

POLITEKNIK STATISTIKA
STIS

BUKU PEDOMAN

MENYUSUN MODUL PRAKTIKUM

dengan paket R `modulstis`

Panduan R Markdown, Quarto, LaTeX, dan Otomatisasi

11 Bab · 5 Lampiran · untuk paket versi 0.3.7

Diperbarui 5 September 2026

Penyusun

Achmad Syahrul Choir

PERNYATAAN HAK CIPTA

Buku Pedoman Menyusun Modul Praktikum dengan Paket R `modulstis`

Hak Cipta © 2026 **Achmad Syahrul Choir dan Politeknik Statistika STIS**

Hak cipta dilindungi undang-undang. Dilarang memperbanyak, menyalin, menggandakan, atau memindahkan sebagian maupun seluruh isi buku ini dalam bentuk dan dengan cara apa pun tanpa izin tertulis dari penyusun dan Politeknik Statistika STIS.

Paket `modulstis` dilisensikan di bawah **MIT License**. Seluruh contoh kode dalam buku ini boleh dipakai dan disesuaikan secara bebas.

Program Studi D-IV Komputasi Statistik
Politeknik Statistika STIS · Jakarta

Kata Pengantar

Buku ini lahir dari pengalaman menyusun dua mata kuliah praktikum: **Analisis Peubah Ganda** dengan dua puluh enam modul, dan **Pemrograman Statistika** dengan empat belas modul. Seluruhnya empat puluh modul, masing-masing beserta kunci jawabannya — delapan puluh dokumen yang harus seragam, sinkron, dan terpelihara.

Pekerjaan itu tidak mungkin dikerjakan dengan cara menyalin naskah. Dari kebutuhan itulah paket `modulstis` dikembangkan, dan dari pengalaman memakainyalah buku ini disusun.

Apa yang membedakan buku ini. Hampir setiap kotak *Kesalahan Umum* di dalamnya berasal dari kekeliruan yang benar-benar terjadi selama penyusunan — bukan kekeliruan yang dibayangkan. Kunci `n` pada YAML yang diam-diam terbaca sebagai boolean sehingga data selalu seratus baris. Penomor halaman yang direset lingkungan `titlepage` tanpa pesan galat apa pun. Versi paket lama yang terpasang di sistem diam-diam menimpa sumber yang sedang disunting. Ketiganya memakan waktu untuk ditemukan, dan ketiganya kini tercatat agar pembaca tidak perlu mengulanginya.

Buku ini ditulis memakai `modulstis` sendiri. Setiap kotak berwarna, setiap penomoran contoh, dan tata letak halaman yang Anda lihat dihasilkan oleh paket yang sedang dijelaskan. Dengan begitu buku ini sekaligus menjadi bukti kemampuannya — dan bila ada yang tidak berfungsi, akan segera ketahuan.

Cara membaca. Bila Anda ingin segera mencoba, kerjakan Bab 2 lalu langsung ke Bab 3; tiga puluh menit sudah cukup untuk menghasilkan modul pertama. Bab 1 boleh dibaca belakangan. Bagi yang mencari perintah tertentu, Lampiran A dan B disusun sebagai rujukan cepat yang dapat dibuka tanpa membaca babnya.

Buku ini tentu masih dapat disempurnakan. Kritik dan saran, khususnya dari rekan yang memakainya untuk menyusun mata kuliah sendiri, sangat diharapkan.

Jakarta, 5 September 2026

Achmad Syahrul Choir
Penyusun

Daftar Isi

Bagian I — Memulai

Bab	Judul	Hlm
1	Mengapa `modulstis`	1
2	Menyiapkan Lingkungan	4
3	Modul Pertama dalam 15 Menit	10

Bagian II — Menyusun Isi

Bab	Judul	Hlm
4	Anatomi Spesifikasi YAML	14
5	Sebelas Bagian Modul	20
6	Contoh, Kode, dan Latihan	25

Bagian III — Sintaks

Bab	Judul	Hlm
7	Sintaks LaTeX yang Dipakai Modul	31
8	Sintaks R Markdown dan Knitr	39

Bagian IV — Menyesuaikan dan Menerbitkan

Bab	Judul	Hlm
9	Menyesuaikan Tampilan	44
10	Menyusun Buku Gabungan	48
11	Alur Kerja Sehari-hari	52

Lampiran

	Judul	Hlm
A	Rujukan Cepat Spesifikasi YAML	55
B	Kartu Referensi LaTeX	59
C	Rujukan API	61
D	Pemecahan Masalah	63
E	Studi Kasus: Mata Kuliah Baru	66

Bagian Akhir

Judul	Hlm
Glosarium	71
Indeks Perintah	73
Daftar Pustaka	75

Cara Membaca Buku Ini

Tiga Jalur Pembacaan

Pilih jalur sesuai kebutuhan.

Kebutuhan	Jalur	Perkiraan waktu
Ingin segera mencoba	Bab 2 → Bab 3	30 menit
Ingin menyusun mata kuliah	Bab 1–6, lalu Lampiran E	satu hari
Mencari perintah tertentu	Lampiran A dan B	seketika

Simbol dan Kotak

Buku ini memakai kotak berwarna dengan peran berbeda. Mengenalinya membantu membaca secara efisien — kotak yang sama juga dipakai di dalam modul praktikum yang akan Anda susun.

Perhatian. Peringatan yang wajib diperhatikan sebelum melanjutkan.

Kesalahan Umum. Kesalahan yang lazim terjadi beserta cara menghindarinya. Sebagian besar berasal dari kekeliruan nyata selama penyusunan modul.

Tips dan Praktik Baik. Praktik baik yang mempermudah pekerjaan atau menjaga mutu hasil.

Catatan. Catatan pelengkap yang perlu diingat.

Kotak info. Tabel acuan dan rujukan cepat.

Langkah. Instruksi bertahap yang dikerjakan berurutan.

Variasi dan Alternatif. Alternatif atau perkakas lain untuk tujuan serupa.

Kesepakatan Penulisan

`teks monospasi`	nama fungsi, berkas, kunci YAML, atau perintah
teks miring	istilah asing atau penekanan
Blok berlatar biru	kode R yang dapat dijalankan
Blok berlatar putih	potongan YAML atau LaTeX
Blok berlatar hijau	keluaran hasil menjalankan kode

Tentang ilustrasi antarmuka. Gambar antarmuka RStudio dalam buku ini adalah **ilustrasi tata letak**, bukan tangkapan layar sesungguhnya. Ilustrasi dipilih agar bagian yang dibahas dapat diberi penanda bernomor dengan jelas. Isi yang ditampilkan diambil dari proyek nyata sehingga tetap mewakili keadaan yang akan Anda jumpai.

Prasyarat Pembaca

Diandaikan sudah bisa	menjalankan R; menulis R Markdown sederhana
Tidak diandaikan	menguasai LaTeX — justru diajarkan pada Bab 7
Tidak diperlukan	pengalaman mengembangkan paket R

Berkas Pendamping

`modulstis_0.3.7.tar.gz`	paket siap pasang
`inst/specs_ps/`	14 spesifikasi Pemrograman Statistika (D-IV Statistika)
`inst/specs_d3/`	14 spesifikasi Pemrograman Statistika (D-III Statistika)
`Sumber_PS.zip`, `Sumber_PS_D3.zip`	seluruh sumber kedua mata kuliah tersebut

Kedua puluh delapan spesifikasi tersebut dapat dipakai sebagai titik awal maupun sebagai contoh penulisan yang sudah teruji. Keduanya sekaligus menjadi contoh penerapan **kedua backend**: spesifikasi D-IV dirender lewat bookdown dan D-III lewat Quarto (Bagian 2.10).

Bab 1 Mengapa modulstis

1.1 Masalah yang Hendak Diselesaikan

Menyusun modul praktikum secara manual tampak sederhana pada modul pertama. Persoalan baru muncul ketika modulnya bertambah menjadi empat belas atau dua puluh enam.

Empat masalah berikut berulang pada hampir semua penyusunan modul.

Preamble yang disalin berulang. Setiap modul memerlukan pengaturan LaTeX yang sama: warna, font, geometri halaman, definisi kotak berwarna, gaya judul. Ketika salinan sudah tersebar di dua puluh berkas, satu perubahan kecil — misalnya menggeser warna oranye — menuntut penyuntingan dua puluh kali. Dalam praktiknya, selalu ada satu atau dua yang terlewat.

Penomoran yang tidak sinkron. Modul memuat penomoran bagian, contoh, kode, dan latihan. Bila satu contoh disisipkan di tengah, seluruh nomor sesudahnya harus digeser, termasuk setiap rujukan silang di dalam teks. Pekerjaan ini menjemukan dan rawan luput.

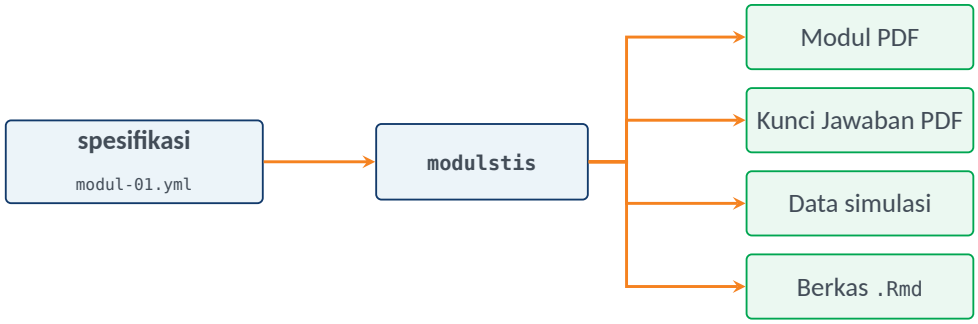
Kunci jawaban yang tercecer. Soal dan solusinya hidup di dua berkas terpisah. Ketika soal diperbaiki tetapi kuncinya lupa disesuaikan, kekeliruan itu baru ketahuan saat penilaian.

Ketidakseragaman antar-modul. Modul yang disusun pada waktu berbeda cenderung berbeda strukturnya: ada yang memuat ringkasan, ada yang tidak; judul bagian tidak sama; gaya penulisan interpretasi berbeda-beda.

> **Akar persoalannya.** Keempat masalah di atas berakar pada satu hal yang sama: **naskah dipakai sebagai sumber kebenaran**. Selama isi modul hidup di dalam berkas naskah yang saling disalin, tidak ada mekanisme yang memaksa keseragaman maupun menjaga kesinkronan.

1.2 Gagasan Pokok

`modulstis` memindahkan sumber kebenaran dari naskah ke **spesifikasi**. Satu modul dideskripsikan sebagai berkas data — bukan naskah — lalu paket membangkitkan seluruh keluarannya.



Gambar 1.1: Satu spesifikasi menghasilkan seluruh keluaran modul

Akibat pergeseran ini, keempat masalah tadi hilang dengan sendirinya:

Masalah	Bagaimana teratasi
Preamble disalin berulang	preamble dibangkitkan dari satu templat; ubah sekali, berlaku semua
Penomoran tidak sinkron	nomor dihitung otomatis saat render
Kunci tercecer	soal dan solusi hidup di **satu** spesifikasi
Tidak seragam	struktur bagian ditetapkan paket, bukan diketik ulang

1.3 Apa yang Dihasilkan

Dari satu berkas spesifikasi, paket menghasilkan empat keluaran sekaligus.

Keluaran satu modul.

Berkas	Isi
`Modul_NN_*.pdf`	modul praktikum siap cetak, bersampul pembatas
`Kunci_NN_*.pdf`	kunci jawaban seluruh latihan dan penugasan
`Modul_NN_*.Rmd`	sumber R Markdown yang dibangkitkan
`data/*.csv`	data simulasi beserta `gen_data.R`

Pada tingkat mata kuliah, paket juga menyusun **buku gabungan** — seluruh modul dirangkai menjadi satu berkas dengan penomoran halaman yang berlanjut, dilengkapi bagian awal dan bagian akhir. Hal ini dibahas pada Bab 10.

1.4 Apa yang Bukan Tujuan Paket Ini

Agar harapan tidak keliru, tiga hal berikut perlu ditegaskan.

`modulstis` **bukan penyunting visual**. Seluruh isi ditulis sebagai teks, sehingga dapat disimpan di kendali versi dan diperbandingkan perubahannya.

`modulstis` **tidak menggantikan** `bookdown`, R Markdown, maupun Quarto. Paket ini mengorkestrasi keduanya — menyiapkan berkas, memanggil mesin render, lalu merapikan hasilnya. Dua backend tersedia dan keduanya mempunyai contoh penerapan penuh: **bookdown** (berkas antara `.Rmd`) diperagakan pada Pemrograman Statistika D-IV, dan **Quarto** (berkas antara `.qmd`) pada Pemrograman Statistika D-III. Berpindah di antara keduanya cukup dengan mengganti argumen `backend`; lihat vignette *Contoh Penerapan modulstis Secara Penuh (Dua Backend)*.

`modulstis` **tidak menulis isi modul untuk Anda**. Materi, contoh, dan interpretasi tetap merupakan pekerjaan penyusun. Yang diotomatiskan adalah bentuk dan penyusunannya, bukan substansinya.

1.5 Peta Buku Ini

Susunan buku.

Bagian I (Bab 1–3)	Memulai: pemasangan hingga modul pertama
Bagian II (Bab 4–6)	Menyusun isi: spesifikasi, bagian modul, contoh dan latihan
Bagian III (Bab 7–8)	Sintaks: LaTeX dan R Markdown
Bagian IV (Bab 9–11)	Menyesuaikan tampilan dan menerbitkan buku
Lampiran	Rujukan cepat, API, pemecahan masalah, studi kasus

Bila Anda ingin segera mencoba, lompatlah ke Bab 3 setelah menyelesaikan pemasangan pada Bab 2. Bab 1 ini boleh dibaca belakangan.

Bab 2 Menyiapkan Lingkungan

Bab ini memastikan komputer Anda siap sebelum modul pertama dibuat. Seluruh langkah hanya perlu dikerjakan sekali.

2.1 Prasyarat Lingkungan

Yang harus tersedia.

R	versi 4.1 atau lebih baru; mesin penjalan seluruh kode
RStudio	penyunting; membuat pekerjaan lebih nyaman meski tidak wajib
XeLaTeX	mesin penyusun PDF; disediakan TinyTeX atau TeX Live
poppler-utils	menyediakan `pdfunite`, `pdftoppm`, `pdfinfo`
Pola pemenggalan	kaidah pemenggalan kata bahasa Indonesia bagi LaTeX
Quarto	*opsional*; hanya bila memakai backend Quarto (lihat Bagian 2.10)

Selain itu diperlukan font **Carlito**, yaitu font bebas yang metriknya setara Calibri dan dipakai pada seluruh tata letak modul.

2.2 Memasang R dan RStudio

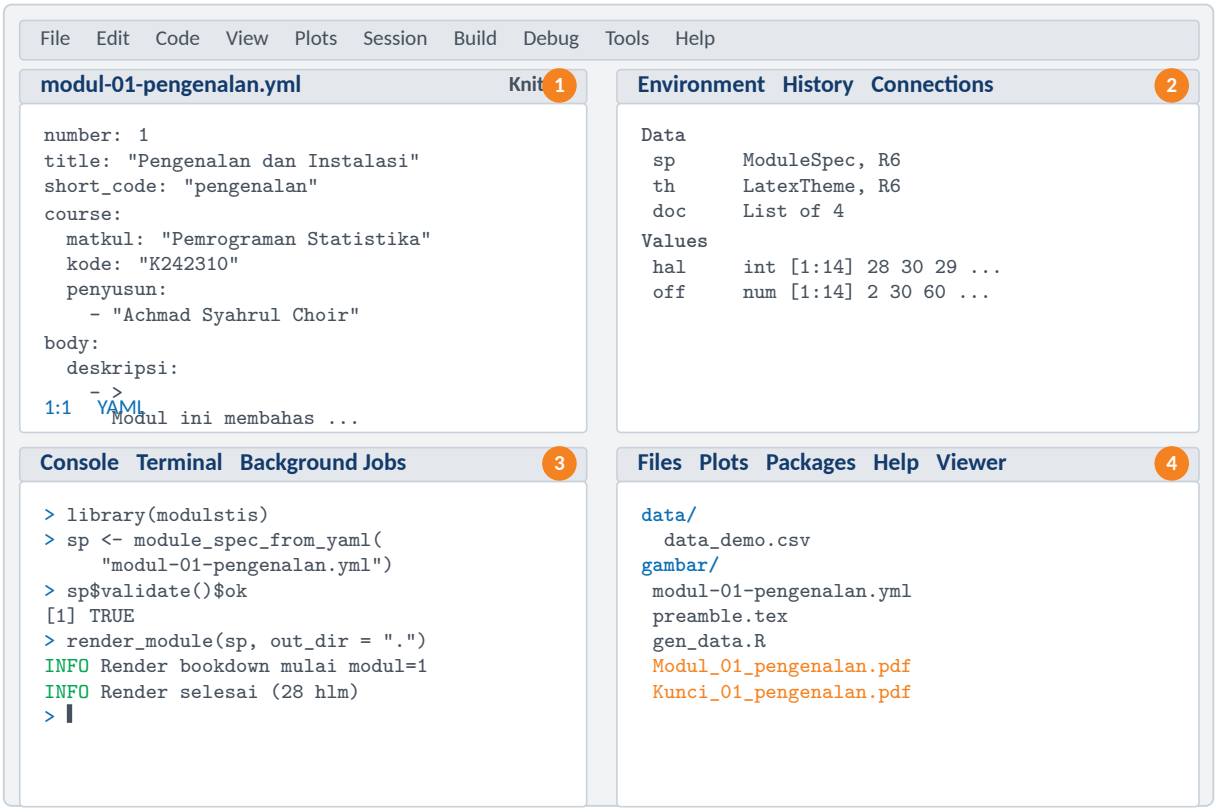
R diunduh dari `cran.r-project.org`, pilih paket sesuai sistem operasi Anda. Pemasangan berjalan seperti aplikasi biasa.

RStudio Desktop diunduh dari `posit.co/download/rstudio-desktop`. Pasang setelah R terpasang, agar RStudio langsung mengenalinya.

Tips dan Praktik Baik. Bila Anda memakai penyunting lain — VS Code, Positron, atau bahkan terminal — seluruh perintah dalam buku ini tetap berlaku. Buku ini sengaja ditulis dalam bentuk perintah yang dapat dijalankan di mana saja, dengan catatan tambahan bila RStudio menyediakan jalan pintas melalui menu.

2.3 Mengenali Tata Letak RStudio

Empat panel RStudio akan disebut berulang kali dalam buku ini. Gambar berikut menandai keempatnya.



Gambar 2.1: Ilustrasi tata letak RStudio saat menyunting spesifikasi modul

Keterangan panel.

1	**Source**	tempat menyunting berkas spesifikasi `.yml`, berkas `.Rmd`, maupun `preamble.tex`
2	**Environment**	memperlihatkan objek yang sedang hidup, misalnya `sp` (spesifikasi) dan `th` (tema)
3	**Console**	tempat menjalankan perintah; pesan `INFO` dari paket muncul di sini saat render berjalan
4	**Files**	memperlihatkan berkas yang dihasilkan; PDF hasil render tampak di sini dan dapat langsung dibuka

Sepanjang buku, rujukan seperti “amati panel Console” mengacu pada penomoran gambar di atas.

2.4 Memasang XeLaTeX

Cara paling ringan adalah **TinyTeX**, distribusi LaTeX kecil yang dapat dipasang dari dalam R dan otomatis menambah paket yang dibutuhkan.

```
install.packages("tinytex")
tinytex::install_tinytex()
```

Pemasangan mengunduh sekitar 100 MB dan berlangsung beberapa menit. Setelah selesai, **tutup dan buka**

kembali RStudio agar jalur perkakas terbaca.

Bila di komputer Anda sudah terpasang TeX Live atau MiKTeX lengkap, langkah ini tidak perlu.

2.5 Memasang Font Carlito

Font Carlito bersifat bebas dan tersedia sebagai bagian dari paket font Google Crosextra.

Linux. Jalankan `sudo apt install fonts-crosextra-carlito` lalu `fc-cache -f`.

Windows dan macOS. Unduh berkas `Carlito-Regular.ttf` beserta variannya, lalu pasang seperti font biasa (klik kanan, *Install*). Setelah itu jalankan ulang RStudio.

Bila font tidak ditemukan. Galat `Package fontspec Error: The font "Carlito" cannot be found` berarti font belum terpasang atau belum terbaca sistem. Pada Linux, periksa dengan `fc-list | grep -i carlito`. Bila tetap tidak ditemukan, Anda dapat menggantinya sementara dengan font lain melalui penyuntingan preamble — caranya dibahas pada Bab 9.

2.6 Memasang Pola Pemenggalan Bahasa Indonesia

Prasyarat ini mudah terlewat karena ketiadaannya **tidak menimbulkan galat**. Modul tetap tersusun, hanya saja kata-katanya terpenggal menurut kaidah bahasa Inggris.

Gejala yang perlu dikenali. Perhatikan kata yang terpotong di ujung baris. Bila Anda menemukan `meny-atakan`, `pengelom-pokan`, atau `peng-hasilkan`, berarti pola pemenggalan Indonesia belum terpasang. Pemenggalan yang benar mengikuti batas suku kata: `me-nyatakan`, `penge-lompokan`, `meng-hasilkan`.

Pemasangannya bergantung pada distribusi LaTeX yang Anda pakai.

```
# TeX Live berbasis Debian atau Ubuntu
sudo apt-get install texlive-lang-other
```

```
# TinyTeX
tinytex::tlmgr_install(c("hyphen-indonesian"))
```

Periksa keberhasilannya dari terminal:

```
kpsewhich hyph-id.tex
```

Bila berkasnya ditemukan, polanya sudah tersedia. Paket `modulstis` memilihnya secara otomatis melalui satu baris pada preamble:

```
\language=\csname l@indonesian\endcsname
```

Tips dan Praktik Baik. Cara ini sengaja dipilih daripada memuat paket `polyglossia` atau `babel`. Selain tidak menambah ketergantungan, ia tidak mengubah penamaan bagian — "Daftar Isi", "Tabel", dan "Gambar" tetap sebagaimana ditetapkan paket — dan tidak bentrok dengan makro sampul.

2.7 Memasang poppler-utils

Perkakas ini dipakai untuk menggabungkan PDF dan membaca jumlah halamannya.

Pemasangan. **Linux:** `sudo apt install poppler-utils`
macOS: `brew install poppler`
Windows: unduh *poppler for Windows*, lalu tambahkan foldernya ke `PATH`

Perkakas ini hanya diperlukan saat menyusun buku gabungan (Bab 10). Untuk membuat modul satuan, ketiadaannya tidak menghalangi.

2.8 Memasang `modulstis`

Paket dipasang dari berkas tarball yang disertakan.

```r
install.packages("modulstis_0.3.7.tar.gz", repos = NULL, type = "source")
```

Bila Anda sedang mengembangkan paketnya sendiri, muat sumbernya langsung tanpa memasang.

```r
pkgload::load_all("paket_R/modulstis")
```

Sumber lokal tertimpa versi terpasang. Bila paket versi lama pernah dipasang di sistem, perintah `library(modulstis)` akan memuat versi lama itu — bukan sumber yang sedang Anda sunting. Akibatnya perubahan Anda seolah tidak berpengaruh. Utamakan `pkgload::load_all()` saat mengembangkan, atau hapus versi terpasang dengan `remove.packages("modulstis")`.

2.9 Daftar Periksa Kesiapan

Jalankan potongan berikut. Seluruh baris harus bernilai `TRUE`.

```r
periksa_kesiapan <- function() {
  hasil <- c(
    R_versi_cukup   = getRversion() >= "4.1.0",
    modulstis       = requireNamespace("modulstis", quietly = TRUE),
    rmarkdown       = requireNamespace("rmarkdown", quietly = TRUE),
    knitr           = requireNamespace("knitr", quietly = TRUE),
    bookdown        = requireNamespace("bookdown", quietly = TRUE),
    xelatex         = nzchar(Sys.which("xelatex")),
    pdfunite        = nzchar(Sys.which("pdfunite")),
    pdfinfo         = nzchar(Sys.which("pdfinfo"))
```

```
  )
  print(hasil)
  if (all(hasil)) message("Lingkungan siap.")
  else message("Belum siap: ", paste(names(hasil)[!hasil], collapse = ", "))
  invisible(hasil)
}
periksa_kesiapan()
```

Keluaran yang diharapkan:

```
  R_versi_cukup       modulstis       rmarkdown           knitr
           TRUE            TRUE            TRUE            TRUE
       bookdown         xelatex        pdfunite         pdfinfo
           TRUE            TRUE            TRUE            TRUE
Lingkungan siap.
```

Font Carlito tidak dapat diperiksa dari R secara lintas sistem; keberhasilannya akan terbukti pada render pertama di Bab 3.

Bila Anda berencana memakai backend Quarto, tambahkan satu pemeriksaan lagi — ini **opsional** dan tidak diperlukan bagi backend bookdown:

```
c(quarto_cli = nzchar(Sys.which("quarto")),
  paket_quarto = requireNamespace("quarto", quietly = TRUE))
```

2.10 Memilih Backend: R Markdown atau Quarto

`modulstis` dapat merender lewat **dua backend**, dan keduanya menghasilkan modul serta kunci yang isinya sama. Yang berbeda hanya berkas antara dan mesin yang memprosesnya.

Dua backend.

	bookdown (bawaan)	**quarto**
Berkas antara	`.Rmd`	`.qmd`
Mesin	`rmarkdown::render()`	Quarto CLI
Prasyarat	sudah terpasang bersama R	Quarto CLI harus dipasang
Mesin PDF	XeLaTeX	XeLaTeX
Preamble BPS	sama	sama

Cara berpindah **hanya satu argumen**:

```
render_module(spec, backend = "bookdown")   # bawaan
render_module(spec, backend = "quarto")     # menuntut Quarto CLI

proj <- ModuleProject$new(specs_dir = "spec", backend = "quarto")
```

Memasang Quarto. Unduh pemasang dari https://quarto.org/docs/get-started/, atau pada Linux berbasis

Debian/Ubuntu pasang berkas `.deb` yang disediakan di sana. RStudio versi baru sudah menyertakan Quarto sehingga sering kali tidak perlu memasang apa pun. Periksa dengan:

```
quarto::quarto_version()
#> [1] '1.6.39'
```

Mana yang sebaiknya dipakai.

- **Pakai bookdown** bila Anda ingin prasyarat sesedikit mungkin, bekerja pada komputer yang tidak dapat memasang perangkat lunak baru, atau menyusun modul yang akan digabung menjadi buku bersama modul lama.
- **Pakai Quarto** bila mata kuliah Anda juga memakai Python atau Julia lewat mesin Quarto, atau bila Anda memang sudah terbiasa dengan `.qmd`.
- **Jangan mencampur keduanya dalam satu buku.** Isi kedua keluaran identik, tetapi jumlah halamannya dapat berbeda beberapa halaman karena Quarto dan rmarkdown menyusun blok kode serta gambar dengan spasi yang sedikit berbeda. Penomoran halaman berlanjut (Bab 10) mengandaikan seluruh modul dirender dengan backend yang sama.

Contoh penerapan penuh untuk **kedua** backend tersedia sebagai vignette *Contoh Penerapan modulstis Secara Penuh (Dua Backend)*: Pemrograman Statistika D-IV dirender lewat bookdown, dan D-III lewat Quarto, dari spesifikasi yang bentuknya sama.

2.11 Galat Pemasangan yang Lazim

Gejala dan tindakan.

Gejala	Tindakan
`xelatex not found`	TinyTeX belum terpasang, atau RStudio belum dijalankan ulang sesudahnya
`! LaTeX Error: File` `` `tcolorbox.sty' not found ``	jalankan `tinytex::tlmgr_install("tcolorbox")`
`fontspec Error: font cannot be found`	font Carlito belum terpasang (lihat bagian sebelumnya)
`pdfunite: command not found`	poppler-utils belum terpasang; hanya menghalangi penyusunan buku gabungan
`there is no package called 'R6'`	pasang dependensi: `install.packages(c("R6","yaml","pur`

2.12 Memulai Proyek Baru

Setelah paket terpasang, kerangka proyek dapat dibuat langsung dari RStudio: **File > New Project > New Directory > Proyek Modul Praktikum (modulstis)**.

RStudio menanyakan identitas mata kuliah — nama, kode, program studi, semester, penyusun, dan banyaknya modul — lalu menyiapkan folder beserta berkas pendukungnya.

Isi proyek yang dihasilkan.

`spec/`	spesifikasi modul dalam YAML, satu berkas per modul
`modul/`	keluaran render: PDF modul, kunci jawaban, dan data
`buku/`	bagian awal dan akhir buku gabungan
`render_modul.R`	skrip render seluruh spesifikasi
`README.md`	identitas mata kuliah dan alur kerjanya

Proyek baru menyertakan satu spesifikasi contoh yang **lolos validasi apa adanya**. Jalankan skrip berikut untuk membuktikan bahwa lingkungan Anda sudah siap sebelum menulis isi sungguhan.

```
Rscript render_modul.R
```

Bila PDF modul dan kunci terbentuk tanpa galat, seluruh prasyarat pada bab ini sudah terpenuhi. Bila gagal, pesan galatnya menunjuk prasyarat mana yang belum tersedia.

Tips dan Praktik Baik. Fungsi yang sama dapat dipanggil dari konsol tanpa melalui menu RStudio:

```
modulstis::buat_proyek_modul("folder_baru", matkul = "Nama Mata Kuliah", kode = "K000000")
```

Bab 3 Modul Pertama dalam 15 Menit

Bab ini mengajak Anda menghasilkan satu modul praktikum lengkap sesegera mungkin. Rinciannya sengaja ditunda ke bab-bab berikutnya — tujuan di sini adalah memperoleh keberhasilan pertama dan melihat wujud keluarannya.

3.1 Menyiapkan Folder Kerja

1 — Buat folder. Buat folder kosong, misalnya `modul-saya`. Di RStudio: menu *File* → *New Project* → *New Directory* → *New Project*. Bekerja dalam proyek membuat jalur berkas relatif selalu benar.

3.2 Menyalin Spesifikasi Contoh

Paket menyertakan empat puluh spesifikasi siap pakai: 26 dari mata kuliah Analisis Peubah Ganda dan 14 dari Pemrograman Statistika. Kita pinjam salah satunya sebagai titik awal.

```r
library(modulstis)

# lihat spesifikasi yang tersedia
list.files(system.file("specs_ps", package = "modulstis"))

# salin satu ke folder kerja
file.copy(
  system.file("specs_ps", "modul-01-pengenalan.yml", package = "modulstis"),
  "modul-saya.yml"
)
```

3.3 Membaca dan Memeriksa Spesifikasi

Sebelum dirender, spesifikasi sebaiknya diperiksa lebih dahulu. Paket menerapkan prinsip *fail-fast*: kekeliruan dilaporkan sebelum render dimulai, bukan setelah menunggu lama.

```r
sp <- module_spec_from_yaml("modul-saya.yml")

sp$validate()$ok        # TRUE bila spesifikasi sah
sp$number               # nomor modul
sp$title                # judul modul
length(sp$body$contoh)  # banyaknya contoh
```

```
[1] TRUE
[1] 1
[1] "Pengenalan dan Instalasi Perangkat Pemrograman Statistika"
[1] 15
```

> **Bila validasi gagal.** Objek `sp$validate()` mengembalikan `ok` bernilai `FALSE` beserta komponen `messages` berisi seluruh kekeliruan sekaligus — bukan satu per satu. Bacalah pesannya, perbaiki berkas YAML, lalu muat ulang.

3.4 Membangkitkan Data

Modul memakai data simulasi yang dibangkitkan dari resep di dalam spesifikasi. Karena resepnya memuat `seed`, hasilnya selalu sama setiap kali dibangkitkan.

```r
generate_module_data(sp, output_dir = ".")
list.files("data")
```

```
[1] "data_demo.csv"
```

3.5 Merender Modul dan Kunci

Inilah langkah intinya. Tiga baris berikut menghasilkan dua berkas PDF.

```
th <- theme_factory()                              # tema BPS bawaan
r  <- renderer_factory("bookdown", theme = th)     # mesin render

r$render(ModuleBuilder$new(theme = th)$build(sp), ".")      # modul
r$render(AnswerKeyBuilder$new(theme = th)$build(sp), ".")   # kunci jawaban
```

Selama render berjalan, panel Console menampilkan pesan seperti berikut.

```
INFO  [10:14:22] Render (bookdown/rmarkdown) modul=1 jenis=modul
INFO  [10:14:51] Render selesai berkas=Modul_01_pengenalan.pdf
```

> **Tips dan Praktik Baik.** Render pertama biasanya paling lama karena LaTeX perlu menyiapkan berkas pendukung. Render berikutnya jauh lebih cepat.

3.6 Memeriksa Hasil

Folder kerja kini berisi berkas berikut.

```
modul-saya/
  modul-saya.yml                  spesifikasi (sumber kebenaran)
  Modul_01_pengenalan.Rmd         R Markdown yang dibangkitkan
  Modul_01_pengenalan.pdf         modul praktikum siap cetak
  Kunci_01_pengenalan.Rmd
  Kunci_01_pengenalan.pdf         kunci jawaban
  preamble.tex                    preamble LaTeX yang dibangkitkan
  data/data_demo.csv              data simulasi
```

Bukalah `Modul_01_pengenalan.pdf`. Anda akan mendapati sampul pembatas berwarna, sebelas bagian bernomor, kotak berwarna, contoh berlabel, dan kode yang sudah dijalankan beserta keluarannya.

> **Berkas .Rmd adalah hasil, bukan sumber.** Berkas `.Rmd` yang muncul **dibangkitkan** dari spesifikasi. Menyuntingnya memang bisa, tetapi perubahan Anda akan **tertimpa** pada render berikutnya. Sunting berkas `.yml`, bukan `.Rmd`.

3.7 Menyunting Isi Modul

Mari lakukan satu perubahan kecil untuk merasakan alur kerjanya. Bukalah `modul-saya.yml` pada panel Source, lalu ubah judulnya.

```
number: 1
title: "Modul Percobaan Saya"
```

Kemudian render ulang dengan perintah yang sama seperti sebelumnya. Buka kembali PDF-nya: judul pada sampul, pada kepala halaman, dan pada daftar isi seluruhnya sudah berubah — cukup dengan menyunting satu baris.

Inilah gagasan pokok yang disebut pada Bab 1. Isi hidup di satu tempat, dan seluruh keluaran mengikutinya.

3.8 Menyusun Fungsi Pintasan

Karena keempat langkah tadi akan sering diulang, sebaiknya dibungkus menjadi satu fungsi. Letakkan di berkas `bangun.R` pada folder kerja Anda.

```
bangun_modul <- function(spec, folder = ".", data = TRUE) {
  sp <- modulstis::module_spec_from_yaml(spec)
  lap <- sp$validate()
  if (!lap$ok) {
    message("Spesifikasi belum sah:")
    print(lap$messages)
    return(invisible(FALSE))
  }
  if (data) modulstis::generate_module_data(sp, output_dir = folder)
  th <- modulstis::theme_factory()
  r  <- modulstis::renderer_factory("bookdown", theme = th)
  r$render(modulstis::ModuleBuilder$new(theme = th)$build(sp), folder)
  r$render(modulstis::AnswerKeyBuilder$new(theme = th)$build(sp), folder)
  message("Selesai: ", sp$title)
  invisible(TRUE)
}

# pemakaian
bangun_modul("modul-saya.yml")
```

Sejak titik ini, membangun ulang modul cukup satu baris.

3.9 Apa Selanjutnya

Anda kini memiliki modul yang berjalan. Langkah berikutnya bergantung pada kebutuhan Anda.

Ke mana selanjutnya.

Ingin mengubah isi modul	Bab 4 (anatomi spesifikasi) dan Bab 5 (sebelas bagian)
Ingin menambah contoh dan latihan	Bab 6
Ingin memakai kotak, rumus, tabel	Bab 7 (sintaks LaTeX)
Ingin mengatur perilaku kode	Bab 8 (R Markdown dan knitr)
Ingin mengubah warna atau sampul	Bab 9
Ingin menyusun buku gabungan	Bab 10

Bab 4 Anatomi Spesifikasi YAML

Bab ini membedah berkas spesifikasi — sumber kebenaran seluruh modul. Setelah bab ini Anda akan mengenali setiap kunci yang boleh ditulis dan tahu di mana meletakkannya.

4.1 Mengapa YAML

Spesifikasi ditulis dalam YAML karena format ini mudah dibaca manusia sekaligus mudah dibaca mesin. Tidak ada tanda kurung berlebih, dan strukturnya dinyatakan oleh **indentasi**.

Tiga bentuk dasar YAML.

Pasangan kunci–nilai	`number: 1`
Daftar	baris berawalan tanda hubung, mis. `- "Pertama"`
Peta bersarang	kunci diikuti blok yang menjorok ke dalam

4.2 Kerangka Utuh Sebuah Spesifikasi

Berikut kerangka lengkap dengan seluruh kunci tingkat atas. Rinciannya dibahas satu per satu pada subbab berikutnya.

```yaml
number: 11                        # nomor modul (wajib)
title: "Pendugaan dan Uji Rata-rata"
short_code: "inferensia_rata"     # dipakai pada nama berkas
topic: "Inferensia Rata-rata"     # dipakai pada kepala halaman
pertemuan: 11                     # pertemuan ke berapa pada RPS
sub_cpmk: >
  Sub-CPMK 11: Mahasiswa mampu ...
packages: [dplyr]                 # paket yang dipakai modul
contoh_title: "Contoh Praktikum dengan R"     # opsional
tambahan_title: "Tambahan: Penyelesaian dengan Fungsi R"   # opsional

course:                           # identitas mata kuliah
  matkul: "Pemrograman Statistika"
  matkul_besar: "PEMROGRAMAN STATISTIKA"
  kode: "K242310"
  semester: "3 (Ganjil)"
  penyusun:
    - "Achmad Syahrul Choir"

datasets:                         # resep data simulasi
  - name: data_pelatihan
    role: contoh
    recipe:
      kind: tabular
```

```yaml
      "n": 60
      seed: 2025
      columns:
        kelompok: {type: category, levels: [Kontrol, Perlakuan]}
        nilai: {type: numeric, min: 50, max: 95, digits: 1}

body:                             # ISI MODUL (dibahas Bab 5)
  deskripsi: [...]
  tujuan: [...]
  # ... sebelas bagian

exercises:                        # latihan & penugasan + solusinya
  - id: "L11.1"
    points: 15
    solution: "Ringkasan solusi"
    judul_r: "Solusi Tingkat Dasar (R)"
    solusi_r: |
      mean(x)
    interpretasi: >
      Penjelasan solusinya ...

references:                       # daftar pustaka
  - "Walpole, R. E., dkk. (2016). Probability and Statistics ..."
```

4.3 Kunci Tingkat Atas

Rujukan kunci tingkat atas.

Kunci	Tipe	Keterangan
`number`	integer	nomor modul; menentukan penomoran bagian `N.1–N`
`title`	teks	judul lengkap; tampil pada sampul dan daftar isi
`short_code`	teks	potongan nama berkas keluaran
`topic`	teks	judul ringkas untuk kepala halaman
`pertemuan`	integer	pertemuan ke berapa pada RPS; tampil di sampul
`sub_cpmk`	teks	rumusan capaian; tampil di sampul
`packages`	daftar	paket yang dipakai; dirujuk bagian *Alat dan Bahan*
`contoh_title`	teks	judul bagian Contoh; kosongkan untuk memakai baw
`tambahan_title`	teks	judul bagian Tambahan
`course`	peta	identitas mata kuliah
`datasets`	daftar	resep data simulasi
`body`	peta	isi kesebelas bagian modul
`exercises`	daftar	solusi latihan dan penugasan
`references`	daftar	daftar pustaka

Tips dan Praktik Baik. Hanya `number`, `title`, dan `body` yang benar-benar wajib. Sisanya memiliki nilai bawaan yang masuk akal — tetapi mengisinya membuat modul jauh lebih rapi.

4.4 Blok `course`: Identitas Mata Kuliah

Blok ini menentukan teks yang muncul pada sampul pembatas dan kepala halaman.

```yaml
course:
  matkul: "Pemrograman Statistika"
  matkul_besar: "PEMROGRAMAN STATISTIKA"
  kode: "K242310"
  semester: "3 (Ganjil)"
  prodi: "D-IV Statistika"
  penyusun:
    - "Achmad Syahrul Choir"
    - "Rudi Salam"
    - "Christiana Anggraeni Putri"
```

Kunci pada `course`.

`matkul`	nama mata kuliah; tampil pada pita sampul
`matkul_besar`	versi huruf kapital untuk judul besar
`kode`	kode mata kuliah
`semester`	mis. `"3 (Ganjil)"`
`prodi`	nama program studi; tampil pada sampul dan kaki halaman. Bawaannya `"D-IV Komputasi Statistik"`
`penyusun`	daftar nama; boleh satu, boleh banyak — seluruhnya dirangkai otomatis dengan koma
`halaman_awal`	nomor halaman pertama modul; dipakai penomoran berlanjut (lihat Bab 10)

Karena identitas ini sama untuk seluruh modul dalam satu mata kuliah, blok `course` biasanya disalin apa adanya antar-spesifikasi.

Tips dan Praktik Baik. Kunci `prodi` memungkinkan satu paket melayani beberapa program studi sekaligus. Kedua himpunan spesifikasi bawaan memperagakannya: `inst/specs_ps/` memakai `"D-IV Statistika"` dan `inst/specs_d3/` memakai `"D-III Statistika"` — isinya berbeda, tetapi bentuk spesifikasinya sama dan cukup satu baris yang disetel.

4.5 Blok `datasets`: Resep Data Simulasi

Data modul tidak disimpan sebagai berkas mentah, melainkan sebagai **resep**. Dengan begitu data dapat dibangkitkan ulang kapan saja dan selalu identik.

```yaml
datasets:
  - name: data_pelatihan          # menjadi data/data_pelatihan.csv
    role: contoh                  # contoh | latihan | penugasan
    recipe:
      kind: tabular
      "n": 60                     # perhatikan tanda kutipnya
      seed: 2025
      columns:
        peserta:  {type: integer, min: 1001, max: 1060}
        kelompok: {type: category, levels: [Kontrol, Perlakuan], probs: [0.5, 0.5]}
        nilai:    {type: numeric, min: 50, max: 95, digits: 1}
```

4.5.1 Jenis kolom yang tersedia

Tipe kolom.

Tipe	Argumen	Menghasilkan
`numeric`	`min`, `max`, `digits`	bilangan pecahan seragam
`integer`	`min`, `max`	bilangan bulat
`category`	`levels`, `probs`	kategori acak menurut peluang
`balanced`	`levels`, `each`	kategori berulang **berimbang**, untuk rancangan faktorial

Tipe `balanced` diperlukan ketika rancangan menuntut jumlah amatan sama di tiap sel — misalnya percobaan faktorial pada modul ANOVA.

```yaml
columns:
  varietas: {type: balanced, levels: [V1, V2, V3], each: 30}
  pupuk:    {type: balanced, levels: [Organik, Anorganik], each: 15}
```

Resep di atas menghasilkan rancangan 3 × 2 dengan 15 amatan per sel.

Kunci `n` harus diberi tanda kutip. Tulislah `"n": 60`, bukan `n: 60`. Dalam YAML 1.1, kunci telanjang `n` terbaca sebagai *boolean* `false` (singkatan dari *no*), sehingga ukuran contoh terabaikan dan paket diam-diam memakai nilai bawaan 100 baris. Kekeliruan ini tidak memunculkan pesan galat — data tetap terbentuk, hanya jumlahnya keliru.

4.6 Blok `references`

Daftar pustaka ditulis sebagai daftar teks biasa. Penulisan gaya APA dianjurkan agar seragam.

```yaml
references:
  - "Walpole, R. E., Myers, R. H., & Ye, K. (2016). *Probability and Statistics
     for Engineers and Scientists* (9th ed.). Pearson."
  - "Wickham, H. (2016). *ggplot2: Elegant Graphics for Data Analysis*. Springer."
```

Tanda bintang berpasangan menghasilkan cetak miring, mengikuti kelaziman Markdown.

4.7 Aturan Penulisan YAML yang Sering Menjebak

Lima jebakan penulisan YAML. **1. Indentasi memakai spasi, bukan tab.** Tab menyebabkan galat pembacaan yang pesannya sering membingungkan.

2. Titik dua di dalam teks harus dikutip. Tulis `title: "Bab 1: Pengantar"` — tanpa kutip, YAML mengira ada kunci baru.

3. Nilai yang mirip boolean. Selain `n`, kata `y`, `yes`, `on`, dan `off` juga tertafsir sebagai boolean. Beri tanda kutip bila yang dimaksud teks.

4. Blok teks panjang. Pakai `>` untuk menggabungkan baris menjadi satu paragraf, atau `|` untuk mempertahankan pergantian barisnya.

5. Garis miring terbalik. Di dalam teks berkutip ganda, tulislah `\\` untuk menghasilkan satu garis miring terbalik LaTeX.

4.7.1 Perbedaan `>` dan `|`

```yaml
# > menggabungkan baris menjadi satu paragraf
interpretasi: >
  Kalimat pertama.
  Masih paragraf yang sama.

# | mempertahankan pergantian baris -- dipakai untuk kode
solusi_r: |
  x <- c(1, 2, 3)
  mean(x)
```

Gunakan `>` untuk prosa dan `|` untuk kode. Tertukar akan membuat kode kehilangan pergantian barisnya sehingga tidak dapat dijalankan.

4.8 Memvalidasi Spesifikasi

Sebelum render, jalankan validasi. Paket melaporkan **seluruh** kekeliruan sekaligus, bukan satu per satu.

```r
sp  <- module_spec_from_yaml("modul-11.yml")
lap <- sp$validate()

lap$ok          # TRUE bila sah
lap$messages    # daftar kekeliruan bila tidak
```

Tips dan Praktik Baik. Biasakan memvalidasi setiap kali menyunting spesifikasi. Validasi berlangsung seketika, sedangkan render memakan waktu puluhan detik — menemukan kekeliruan lebih awal menghemat banyak waktu.

Bab 5 Sebelas Bagian Modul

Setiap modul memiliki struktur baku yang sama. Keseragaman ini disengaja: mahasiswa tahu di mana mencari apa, dan penyusun tidak perlu memikirkan ulang kerangka setiap kali.

5.1 Peta Kesebelas Bagian

Urutan baku.

N.1	`deskripsi`	apa isi modul dan mengapa penting
N.2	`tujuan`	capaian yang diharapkan
N.3	`alat`	perangkat lunak, paket, dan berkas data
N.4	`instruksi`	cara mengerjakan dan mengumpulkan
N.5	`teori`	landasan ringkas beserta rumus
N.6	`contoh`	contoh terpandu beserta kode
N.7	`latihan`	latihan berjenjang
N.8	`penugasan`	tugas terstruktur
N.9	`tambahan`	*opsional*; dilewati bila kosong
N.10	`ringkasan`	daftar periksa dan kartu referensi
N.11	`referensi`	daftar pustaka modul

Nomor bagian dihitung otomatis. Bila bagian `tambahan` tidak diisi, `ringkasan` menjadi `N.9` — penomoran menyesuaikan sendiri.

5.2 Bentuk Umum Pengisian

Setiap bagian diisi sebagai **daftar**. Tiap unsur daftar menjadi satu paragraf, satu blok LaTeX, atau satu contoh.

```yaml
body:
  deskripsi:
    - >
      Paragraf pertama ditulis dengan tanda lebih besar sehingga
      barisnya digabung menjadi satu paragraf.
    - >
      Paragraf kedua.
    - "\\begin{kotakperhatian}\nIsi kotak.\n\\end{kotakperhatian}"
```

Tiga jenis unsur.

Prosa Markdown	teks biasa; boleh memakai `**tebal**`, `*miring*`, `` `kode` ``, tabel, dan daftar
Blok LaTeX mentah	teks berawalan `\\begin{...}`; dipakai untuk kotak berwarna
Peta contoh	khusus bagian `contoh` dan `tambahan`; dibahas Bab 6

5.3 N.1 Deskripsi Modul

Menjawab dua pertanyaan: **apa** yang dipelajari dan **mengapa** itu penting. Panjangnya biasanya tiga sampai lima paragraf.

```
deskripsi:
  - >
    Modul ini membahas pendugaan parameter dan uji hipotesis rata-rata.
    Sampai modul sebelumnya kita hanya menggambarkan data yang ada di
    tangan; mulai modul ini kita menarik kesimpulan tentang populasi.
  - >
    Materi dibahas dari dasar hingga lanjut: pendugaan titik, pendugaan
    selang, kerangka uji hipotesis, hingga ukuran pengaruh.
  - "\\begin{kotakperhatian}[Tentang data]\nBerkas data pada modul ini adalah
     \\textbf{data simulasi}, bukan data resmi.\n\\end{kotakperhatian}"
```

Tips dan Praktik Baik. Akhiri Deskripsi dengan kotak *Perhatian* yang menegaskan status data. Penegasan ini diulang pada setiap modul agar mahasiswa tidak keliru menganggap angkanya nyata.

5.4 N.2 Tujuan Praktikum

Ditulis sebagai daftar bernomor dengan **kata kerja operasional** yang terukur.

```
tujuan:
  - "Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:"
  - "1. **Menghitung** pendugaan titik untuk rata-rata dan ragam."
  - "2. **Menyusun** dan **menafsirkan** pendugaan selang."
  - "3. **Melakukan** uji t satu sampel, dua sampel, dan berpasangan."
```

Kata kerja dicetak tebal agar capaiannya menonjol saat dibaca sekilas.

5.5 N.3 Alat dan Bahan

Menyebutkan perangkat lunak, paket, dan berkas data yang diperlukan.

```
alat:
  - "Perangkat dan bahan yang diperlukan:"
  - "- **R** (RStudio); modul ini fokus pada R sesuai RPS."
  - "- Paket: `dplyr` (pasang dengan `install.packages(\"dplyr\")`)."
  - "- Berkas data: `data/data_pelatihan.csv`."
```

5.6 N.4 Instruksi Pengerjaan

Mengatur cara kerja: apa yang disiapkan sebelum sesi, apa yang dikerjakan saat sesi, dan bagaimana pengumpulannya.

```
instruksi:
  - "Ikuti tahapan berikut:"
  - "**Sebelum praktikum:** siapkan direktori kerja; letakkan berkas data
     pada subfolder `data/`."
  - "**Selama praktikum:** kerjakan Contoh berurutan; untuk setiap
     pengujian, **tuliskan hipotesisnya lebih dahulu**."
  - "**Pembagian kerja:** Contoh dan Latihan dikerjakan di kelas;
     **Penugasan dikerjakan di luar sesi**.\n\n**Pengumpulan:** susun
     laporan R Markdown dan kumpulkan melalui Google Classroom."
```

Tips dan Praktik Baik. Hindari menyebut durasi dalam menit. Alokasi waktu berbeda antar-kelas dan antar-semester, sehingga penyebutan durasi cepat menjadi usang.

5.7 N.5 Teori Singkat

Bagian terpanjang setelah Contoh. Susunan yang dianjurkan: kaitan dengan materi sebelumnya, lalu konsep, lalu rumus, lalu peringatan kekeliruan yang lazim.

```
teori:
  - "### Kaitan dengan Materi Sebelumnya"
  - >
    Modul ini memakai langsung bekal Modul 7. Sebaran normal dan
    keluarga fungsi d/p/q/r kini dipakai menghitung nilai kritis.
  - "### Pendugaan Titik"
  - >
    Pendugaan titik adalah satu angka yang dipakai menduga parameter.
  - "\\begin{kotakrumus}\n$SE = \\dfrac{s}{\\sqrt{n}}$\n\\end{kotakrumus}"
  - "\\begin{kotaksalah}[Salah menafsirkan selang]\nSelang 95 persen bukan
     berarti ada peluang 95 persen parameter berada di dalamnya.
     \n\\end{kotaksalah}"
```

Judul berawalan `###` menghasilkan subbagian bernomor `N.5.1`, `N.5.2`, dan seterusnya.

Tips dan Praktik Baik. Membuka Teori dengan subbagian *Kaitan dengan Materi Sebelumnya* sangat membantu mahasiswa melihat modul sebagai rangkaian, bukan potongan yang terpisah-pisah.

5.8 N.6 Contoh Praktikum

Bagian inti modul. Karena strukturnya paling rumit, seluruh Bab 6 dikhususkan untuk membahasnya.

5.9 N.7 Latihan Mandiri

Latihan disusun berjenjang. Bagian ini dibahas bersama Contoh pada Bab 6.

5.10 N.8 Penugasan

Tugas terstruktur yang dikerjakan di luar sesi, sebaiknya berlatar kasus statistik resmi.

```
penugasan:
  - "\\begin{kotaktugas}\nSebagai calon analis \\emph{official statistics},
     susun laporan inferensia atas data pelatihan.\n\\end{kotaktugas}"
  - "**Konteks.** Sebuah unit menyelenggarakan pelatihan bagi 60 peserta ..."
  - "\\tugas{Pendugaan}"
  - "1. Hitung pendugaan titik beserta galat bakunya."
  - "\\tugas{Laporan}"
  - "2. Susun laporan R Markdown berisi kode, keluaran, dan interpretasi."
```

Perintah `\tugas{...}` menghasilkan sub-judul bernomor otomatis di dalam Penugasan.

5.11 N.9 Tambahan (Opsional)

Bagian ini **hanya muncul bila diisi**. Pada modul inferensia, bagian ini dipakai untuk menempatkan penyelesaian dengan fungsi bawaan R — setelah Penugasan — sehingga bagian Contoh dapat berfokus pada perhitungan manual.

```
tambahan:
  - >
    Seluruh perhitungan pada bagian Contoh sengaja dikerjakan manual agar
    rumusnya dipahami. Bagian ini menyajikan penyelesaian yang sama dengan
    fungsi bawaan R. Kerjakan setelah Penugasan selesai.
  - contoh: "Uji t dengan fungsi bawaan"
    kode_r: "t.test(x, mu = 70)"
    judul_r: "t.test satu contoh"
    eval: true
    penjelasan: >
      Fungsi t.test menerima vektor data dan nilai acuan pada argumen mu.
    interpretasi: >
      Angkanya sama persis dengan hasil manual pada Contoh 11.8.
```

Judulnya dapat diganti melalui kunci `tambahan_title` di tingkat atas.

5.12 N.10 Ringkasan Modul

Berisi daftar periksa penguasaan dan kartu referensi cepat.

```
ringkasan:
  - "\\begin{kotaktujuan}\n\\textbf{Daftar periksa penguasaan:}
     \n\\begin{itemize}
     \n\\item[$\\square$] Saya dapat menghitung galat baku.
     \n\\item[$\\square$] Saya dapat menyusun selang kepercayaan.
     \n\\end{itemize}\n\\end{kotaktujuan}"
```

```
    - "**Kartu Referensi Cepat.**"
    - "\\begin{kotakrumus}\n\\begin{tabular}{@{}ll@{}}
       \n\\textbf{Keperluan} & \\textbf{Perintah}\\\\
       \nGalat baku & \\texttt{sd(x)/sqrt(length(x))}\\\\
       \n\\end{tabular}\n\\end{kotakrumus}"
    - "Modul ini membuka rangkaian inferensia ... Modul berikutnya membahas ..."
```

Paragraf penutup sebaiknya menyebut kaitannya dengan modul berikutnya, sehingga rangkaian modul terasa menyambung.

5.13 N.11 Referensi

Diisi otomatis dari kunci `references` di tingkat atas. Bagian ini tidak perlu ditulis di dalam `body`.

5.14 Ringkasan Bab

Yang perlu diingat.	Struktur tetap	sebelas bagian, urutannya ditetapkan paket
	Penomoran otomatis	`N.1` sampai `N.11` mengikuti `number`
	Bagian opsional	hanya `tambahan`; dilewati bila kosong
	Isi berupa daftar	tiap unsur menjadi paragraf, kotak, atau contoh
	Subbagian	judul berawalan `###` menghasilkan `N.x.y`

Bab 6 Contoh, Kode, dan Latihan

Bab ini membahas dua bagian yang paling banyak memakan waktu penyusunan: **Contoh Praktikum** dan **Latihan Mandiri**, beserta kunci jawabannya.

6.1 Struktur Baku Sebuah Contoh

Setiap contoh mengikuti alur lima langkah yang sama. Keseragaman ini membuat mahasiswa cepat terbiasa membacanya.



Gambar 6.1: Alur baku setiap contoh praktikum

Peran tiap langkah.

Judul	menyatakan apa yang dikerjakan contoh ini
Kode	perintah yang akan dijalankan
Output	keluaran sungguhan hasil menjalankan kode
Penjelasan Kode	*apa yang dikerjakan kode* — baris demi baris, argumen demi argumen
Interpretasi Output	*apa arti hasilnya* — kesimpulan dan kaitannya dengan konteks

Dua hal yang berbeda. Penjelasan Kode dan Interpretasi Output sering tercampur ketika ditulis tergesa-gesa. Kalimat "`glimpse` menampilkan tipe tiap kolom" menjelaskan **kode**; kalimat "kolom provinsi bertipe teks sehingga tidak dapat dirata-ratakan" menafsirkan **keluaran**. Keduanya adalah keterampilan berbeda dan sebaiknya dipisahkan.

6.2 Menulis Satu Contoh

Sebuah contoh ditulis sebagai peta dengan enam kunci.

```
contoh:
  - contoh: "Pendugaan titik dan galat baku"
    kode_r: "pel <- read.csv(\"data/data_pelatihan.csv\")\nx <- pel$nilai_post\nc(rata = mean(x), se = sd(x)/sqrt
    judul_r: "mean, sd, dan galat baku"
    eval: true
    penjelasan: >
      Data dibaca lalu kolom nilai disimpan ke peubah x agar tidak ditulis
      berulang. Galat baku dihitung sesuai rumusnya, yaitu simpangan baku
      dibagi akar banyaknya amatan.
    interpretasi: >
      Perhatikan bahwa galat baku jauh lebih kecil daripada simpangan baku:
```

```
        simpangan baku mengukur keragaman antar-amatan, sedangkan galat baku
        mengukur keragaman rata-rata contoh antar-penarikan contoh.
```

Kunci pada sebuah contoh.

Kunci	Wajib	Keterangan
`contoh`	ya	judul contoh; muncul sebagai *Contoh N.x*
`kode_r`	ya[1]	kode R yang dijalankan
`judul_r`	tidak	label pada kotak kode
`kode_py`	—	kode Python (modul dwibahasa)
`judul_py`	—	label kotak kode Python
`eval`	tidak	`true` menjalankan kode; `false` hanya menampilkan
`penjelasan`	tidak	isi kotak *Penjelasan Kode*
`interpretasi`	tidak	isi kotak *Interpretasi Output*

6.3 Menulis Kode di dalam YAML

Kode ditulis sebagai teks berkutip ganda dengan `\n` sebagai penanda baris baru.

```yaml
kode_r: "x <- c(1, 2, 3)\nmean(x)"
```

Cara ini ringkas tetapi menyulitkan bila kodenya panjang. Untuk kode panjang, gunakan blok literal.

```yaml
kode_r: |
  x <- c(1, 2, 3)
  y <- c(4, 5, 6)
  cor(x, y)
```

Tanda kutip di dalam kode. Bila kode memuat tanda kutip ganda — misalnya `read.csv("data/x.csv")` — maka di dalam teks berkutip ganda tanda tersebut harus ditulis `\"`. Blok literal dengan `|` tidak memerlukan pelolosan ini, sehingga lebih aman untuk kode yang banyak memuat teks.

6.4 Contoh Dwibahasa

Pada modul yang mengajarkan R dan Python berdampingan, satu contoh dapat memuat keduanya. Keduanya ditampilkan berurutan dengan label bahasa masing-masing.

```yaml
  - contoh: "Membaca berkas CSV"
    kode_r: "df <- read.csv(\"data/demo.csv\")\nhead(df, 3)"
    judul_r: "read.csv di R"
    kode_py: "import pandas as pd\ndf = pd.read_csv('data/demo.csv')\nprint(df.head(3))"
    judul_py: "read_csv di pandas"
    eval: true
```

[1] Salah satu dari `kode_r` atau `kode_py` harus ada.

```
    penjelasan: >
      Kedua perintah membaca berkas CSV menjadi tabel. Jalur berkas ditulis
      relatif terhadap direktori kerja.
    interpretasi: >
      Keluaran keduanya setara. Perbedaannya, pandas menampilkan indeks baris
      di kolom paling kiri.
```

6.5 Mengendalikan Eksekusi dengan `eval`

Tidak semua kode dapat atau perlu dijalankan.

Kapan memakai `eval: false`.

Keadaan	Alasan
Kode aplikasi Shiny	aplikasi berjalan interaktif, tidak menghasilkan kelua tak
Perintah pemasangan paket	tidak perlu dijalankan ulang saat render
Kode yang memakan waktu lama	memperlambat render tanpa manfaat
Kode yang sengaja salah	dipakai memperagakan galat

Tips dan Praktik Baik. Bila kode tidak dijalankan, sediakan penggantinya agar mahasiswa tetap memperoleh umpan balik: sertakan tangkapan layar hasilnya, atau tampilkan pemeriksaan objek yang *dapat* dijalankan — misalnya `class(ui)` untuk memastikan kerangka aplikasi Shiny sudah sah.

6.6 Menyisipkan Kotak di antara Contoh

Unsur daftar pada bagian `contoh` tidak harus selalu berupa peta contoh. Teks biasa akan tampil sebagai paragraf atau kotak, sehingga dapat dipakai menyisipkan penjelasan di tengah rangkaian contoh.

```
  contoh:
    - contoh: "Contoh pertama"
      kode_r: "..."
    - "\\begin{kotaktips}\nSebelum melanjutkan, jalankan ulang contoh di atas
       dengan nilai berbeda.\n\\end{kotaktips}"
    - contoh: "Contoh kedua"
      kode_r: "..."
```

6.7 Menyusun Latihan Berjenjang

Latihan disusun tiga tingkat. Penjenjangan ini membuat mahasiswa dengan kesiapan berbeda sama-sama memperoleh tantangan yang sesuai.

Tiga tingkat latihan.

Tingkat	Bentuk	Tujuan
Dasar	melengkapi kode yang dikosongkan	memastikan sintaks dikuasai
Menengah	menulis kode sendiri; membetulkan kode salah	melatih kemandirian dan ketelitian
Lanjut	menyelesaikan kasus kecil	melatih perangkaian keterampilan

```
latihan:
  - "Kerjakan bertahap. Gunakan `data/data_pelatihan.csv`."
  - "#### Tingkat Dasar (melengkapi kode)"
  - "\\latihan{Pendugaan titik}"
  - "```{r eval=FALSE}\nx <- pel$nilai_post\nmean(x)\n____(x)   # simpangan baku\n```"
  - "#### Tingkat Menengah (menulis sendiri)"
  - "\\latihan{Uji dua kelompok}"
  - "Uji apakah rata-rata berbeda antara kedua kelompok. Tuliskan hipotesis,
     sajikan ringkasan, jalankan uji, lalu simpulkan."
```

Perintah `\latihan{...}` menghasilkan judul latihan bernomor otomatis.

6.8 Latihan Membetulkan Kode

Jenis latihan ini terbukti sangat efektif karena menyasar miskonsepsi secara langsung. Susunannya: kode yang sengaja salah, lalu **keluaran yang benar** sebagai pembanding.

```
  - "\\latihan{Membetulkan kode yang salah}"
  - "Kode berikut memuat beberapa kesalahan (nama argumen keliru dan
     derajat bebas salah). Temukan dan perbaiki."
  - "```{r eval=FALSE}\n# (a) nama argumen nilai acuan keliru\nt.test(x, mean = 70)\n\n# (b) derajat bebas pada r
  - "**Keluaran yang benar.** Setelah diperbaiki:\n\n```\n# (a) argumen yang benar adalah mu\nt.test(x, mu = 70)\n
```

Tips dan Praktik Baik. Sertakan *Keluaran yang benar* agar mahasiswa dapat memverifikasi perbaikannya sendiri sebelum melihat kunci jawaban. Bila keluarannya berupa gambar, gantikan dengan pemeriksaan objek yang dapat dijalankan — misalnya `class(p)` dan `length(p$layers)` untuk grafik `ggplot2`.

6.9 Menulis Kunci Jawaban

Solusi tidak ditulis di dalam `body`, melainkan pada blok `exercises` di tingkat atas. Dengan begitu soal dan solusinya hidup dalam satu berkas dan tidak mungkin tercecer.

```
exercises:
  - id: "L11.1"
```

```
  points: 15
  solution: "Tingkat Dasar: sd, sqrt, dan argumen conf.level."
  judul_r: "Solusi Tingkat Dasar (R)"
  solusi_r: |
    pel <- read.csv("data/data_pelatihan.csv")
    x <- pel$nilai_post
    mean(x)
    sd(x)                       # simpangan baku
    sd(x) / sqrt(length(x))     # galat baku
  interpretasi: >
    Galat baku dihitung sebagai simpangan baku dibagi akar banyaknya amatan.
    Argumen conf.level mengatur tingkat keyakinan selang.
```

Kunci pada sebuah solusi.

`id`	penanda butir, mis. `"L11.1"` untuk latihan, `"T11.1"` untuk penugasan
`points`	bobot nilai; tampil pada kotak kunci
`solution`	ringkasan satu kalimat tentang isi solusi
`judul_r`	label kotak kode solusi
`solusi_r`	kode solusi (gunakan blok literal `|`)
`solusi_py`	solusi Python bila diperlukan
`eval`	`false` bila solusi tidak boleh dijalankan
`interpretasi`	penjelasan mengapa solusinya demikian

Tips dan Praktik Baik. Isi `interpretasi` pada solusi bukan sekadar mengulang jawabannya, melainkan menjelaskan *alasannya*. Bagian inilah yang dipakai dosen sebagai acuan menilai jawaban mahasiswa yang caranya berbeda tetapi tetap benar.

6.10 Penomoran Otomatis

Seluruh penomoran dihitung paket saat render. Anda tidak perlu — dan sebaiknya tidak — menuliskannya sendiri.

Yang dinomori otomatis.

Contoh N.x	bertambah setiap kunci `contoh` baru
Kode N.x	bertambah setiap blok kode
Latihan N.x	bertambah setiap perintah `\latihan{}`
Tugas N.x	bertambah setiap perintah `\tugas{}`
Bagian N.x	mengikuti urutan sebelas bagian

Karena itu, menyisipkan contoh baru di tengah tidak menuntut penyesuaian apa pun — seluruh nomor

sesudahnya menyesuaikan sendiri saat render berikutnya.

6.11 Ringkasan Bab

Yang perlu diingat.

Alur baku contoh	Judul → Kode → Output → Penjelasan Kode → Interpretasi
Dua kotak berbeda peran	Penjelasan menerangkan kode; Interpretasi menafsirkan hasil
Kode panjang	pakai blok literal `|` agar tanda kutip tidak perlu diloloskan
Latihan berjenjang	Dasar, Menengah, Lanjut
Membetulkan kode	sertakan *Keluaran yang benar*
Solusi	ditulis pada `exercises`, bukan di dalam `body`
Penomoran	seluruhnya otomatis

Bab 7 Sintaks LaTeX yang Dipakai Modul

Bab ini adalah rujukan lengkap perintah LaTeX yang tersedia di dalam modul. Anda tidak perlu menguasai LaTeX secara umum — cukup mengenali perintah yang dibahas di sini.

7.1 Kapan Markdown, Kapan LaTeX

Isi modul dapat ditulis dengan dua cara. Memilih yang tepat menghemat banyak waktu.

Memilih cara penulisan.		
	Pakai Markdown	paragraf biasa, penekanan, daftar, tabel sederhana, judul subbagian
	Pakai LaTeX	kotak berwarna, rumus matematika, tabel di dalam kotak, penataan halaman

Markdown lebih ringkas dan lebih tahan salah tulis. Gunakan LaTeX hanya ketika Markdown tidak menyediakan yang Anda perlukan.

```
# Markdown -- cukup untuk kebanyakan keperluan
- "Paragraf biasa dengan **tebal**, *miring*, dan `kode`."
- "| Kolom A | Kolom B |\n|---------|---------|\n| 1 | 2 |"

# LaTeX -- untuk kotak berwarna
- "\\begin{kotaktips}\nIsi kotak.\n\\end{kotaktips}"
```

7.2 Katalog Kotak

Modul menyediakan empat belas jenis kotak. Berikut seluruhnya beserta wujud aslinya.

7.2.1 Kotak berjudul dapat diubah

Sembilan kotak berikut menerima argumen opsional berupa judul di dalam kurung siku.

```
- "\\begin{kotakperhatian}[Judul Khusus]\nIsi kotak.\n\\end{kotakperhatian}"
```

Bila argumen dikosongkan, judul bawaannya dipakai.

Perhatian. Kotak `kotakperhatian` — judul bawaan *Perhatian*. Dipakai untuk peringatan yang wajib diperhatikan, termasuk penegasan status data simulasi.

Kesalahan Umum. Kotak `kotaksalah` — judul bawaan *Kesalahan Umum*. Dipakai menandai miskonsepsi yang lazim beserta cara menghindarinya.

Tips dan Praktik Baik. Kotak `kotaktips` — judul bawaan *Tips dan Praktik Baik*. Berisi anjuran yang mempermudah pekerjaan atau menjaga mutu hasil.

Variasi dan Alternatif. Kotak `kotakvariasi` — judul bawaan *Variasi dan Alternatif*. Menyebutkan perkakas atau pendekatan lain untuk tujuan serupa.

Catatan. Kotak `kotakcatatan` — judul bawaan *Catatan*. Dipakai pula untuk kotak *Padanan dari R* pada modul Python.

Kotak info dengan judul. Kotak `kotakinfo` — **tanpa** judul bawaan. Cocok memuat tabel acuan. Bila argumen judul dikosongkan, kotak tampil tanpa judul sama sekali.

Penjelasan Kode. Kotak `kotakpenjelasan` — judul bawaan *Penjelasan Kode*. Diisi otomatis dari kunci `penjelasan` pada sebuah contoh; jarang ditulis manual.

Interpretasi Output

Kotak `kotakinterpretasi` — judul bawaan *Interpretasi Output*. Diisi otomatis dari kunci `interpretasi`.

7.2.2 Kotak tanpa argumen judul

Lima kotak berikut tidak menerima argumen.

Kotak `kotakrumus` — memuat rumus atau kerangka perintah:

$$SE = \frac{s}{\sqrt{n}}$$

Kotak `kotaktujuan` — dipakai untuk daftar periksa penguasaan:

- ☐ Butir pertama
- ☐ Butir kedua

Kotak `kotaktugas` — memuat rumusan penugasan yang harus dikerjakan mahasiswa.

Kunci & Pembahasan

Kotak `kotakkunci` — dipakai pada buku kunci jawaban; memuat butir soal beserta bobot nilainya.

Kotak `kotakistilah` dipakai bagian daftar istilah, dan `kotakanatomi` dipakai khusus modul RShiny untuk memperagakan pasangan `ui-server` berwarna serasi.

7.2.3 Tabel rujukan cepat

Empat belas kotak.

Nama	Judul?	Peran
`kotakperhatian`	ya	peringatan wajib
`kotaksalah`	ya	kesalahan umum
`kotaktips`	ya	praktik baik
`kotakvariasi`	ya	alternatif
`kotakcatatan`	ya	catatan, padanan bahasa
`kotakinfo`	ya	tabel acuan
`kotakpenjelasan`	ya	penjelasan kode (otomatis)
`kotakinterpretasi`	ya	interpretasi keluaran (otomatis)
`kotakanatomi`	ya	pasangan ui–server
`kotakrumus`	tidak	rumus
`kotaktujuan`	tidak	daftar periksa
`kotaktugas`	tidak	rumusan tugas
`kotakkunci`	tidak	butir kunci jawaban
`kotakistilah`	tidak	daftar istilah

Markdown tidak berlaku di dalam kotak. Isi kotak diteruskan ke LaTeX apa adanya, sehingga `**tebal**` akan tercetak sebagai bintang, bukan huruf tebal. Di dalam kotak, pakailah perintah LaTeX: `\textbf{tebal}`, `\emph{miring}`, dan `\texttt{kode}`.

7.3 Perintah Penomoran

Empat perintah berikut menghasilkan judul bernomor otomatis.

Perintah penomoran.

Perintah	Menghasilkan	Dipakai di
`\contoh{judul}`	*Contoh N.x*	bagian Contoh (otomatis)
`\kode{judul}`	*Kode N.x*	label kotak kode (otomatis)
`\latihan{judul}`	*Latihan N.x*	bagian Latihan (manual)
`\tugas{judul}`	*Tugas N.x*	bagian Penugasan (manual)

Dua yang pertama dihasilkan paket dari kunci `contoh` dan `judul_r`. Dua yang terakhir Anda tulis sendiri di dalam daftar `latihan` dan `penugasan`.

```
  - "\\latihan{Melengkapi kode dasar}"
  - "\\tugas{Pendugaan parameter}"
```

7.4 Rumus Matematika

Rumus ditulis dengan sintaks LaTeX baku.

Dua bentuk penulisan.

Sebaris	`$...$` — menyatu dengan teks
Terpisah	`\[...\]` — baris tersendiri, di tengah

Contoh penulisan dan hasilnya:

```
Galat baku dirumuskan $SE = s/\sqrt{n}$, sehingga
\[ \bar{x} \pm t_{\alpha/2,\,n-1}\,\frac{s}{\sqrt{n}} \]
```

Galat baku dirumuskan $SE = s/\sqrt{n}$, sehingga

$$\bar{x} \pm t_{\alpha/2,\,n-1}\,\frac{s}{\sqrt{n}}$$

7.4.1 Simbol yang sering dipakai

Rujukan simbol statistika.

Perintah	Hasil	Perintah	Hasil
`\bar{x}`	$\bar{x}$	`\hat{p}`	$\hat{p}$
`\mu`	μ	`\sigma`	σ
`\alpha`	α	`\beta`	β
`\chi^{2}`	χ^{2}	`\eta^{2}`	η^{2}
`\sum`	$\sum$	`\sqrt{x}`	$\sqrt{x}$
`\frac{a}{b}`	$\frac{a}{b}$	`\dfrac{a}{b}`	$\dfrac{a}{b}$
`\leq`	$\leq$	`\neq`	$\neq$
`\times`	$\times$	`\cdot`	$\cdot$
`x_{i}`	x_{i}	`x^{2}`	x^{2}

Tips dan Praktik Baik. Gunakan `\dfrac` alih-alih `\frac` pada rumus sebaris agar pecahannya tidak tercetak terlalu kecil.

7.4.2 Rumus di dalam kotak

Rumus paling sering ditempatkan di dalam `kotakrumus`. Perhatikan pelolosan garis miring terbalik saat ditulis di dalam YAML.

```
- "\\begin{kotakrumus}\n\\textbf{Statistik uji t:}\\\\[3pt]\n
  $t = \\dfrac{\\bar{x} - \\mu_0}{s/\\sqrt{n}}$, \\quad db $= n-1$\n
  \\end{kotakrumus}"
```

Hasilnya:

Statistik uji t:

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}, \quad \text{db} = n - 1$$

Jumlah garis miring terbalik. Di dalam teks YAML berkutip ganda, satu garis miring terbalik LaTeX ditulis `\\`, dan pergantian baris LaTeX (`\\`) ditulis `\\\\`. Bila rumus tidak tampil sebagaimana mestinya, periksalah jumlah garis miring terbaliknya lebih dahulu.

7.5 Tabel

Tabel sederhana cukup ditulis dengan Markdown.

```
| Peubah | Tipe | Keterangan |
|--------|------|------------|
| umur   | integer | umur kepala rumah tangga |
```

Untuk tabel **di dalam kotak**, Markdown tidak berlaku sehingga harus memakai `tabular`.

```
- "\\begin{kotakinfo}[Tipe kolom]\n\\begin{tabular}{@{}lll@{}}\n
   \\textbf{Tipe} & \\textbf{Argumen} & \\textbf{Hasil}\\\\\n
   \\texttt{numeric} & min, max & pecahan\\\\\n
   \\texttt{integer} & min, max & bilangan bulat\\\\\n
   \\end{tabular}\n\\end{kotakinfo}"
```

Penentu kolom `tabular`.

`l`	rata kiri, lebar mengikuti isi
`c`	rata tengah
`r`	rata kanan
`p{4cm}`	lebar tetap, teks membungkus ke bawah
`@{}`	menghapus spasi tepi; letakkan di awal dan akhir

Tips dan Praktik Baik. Bila kolom terakhir memuat kalimat panjang, pakailah `p{...}` agar teksnya membungkus. Tanpa itu, tabel akan melebar melewati batas halaman.

7.6 Gambar

Gambar disisipkan dengan `\includegraphics`, biasanya dibungkus lingkungan `center`.

```
- "\\begin{center}\n\\includegraphics[width=0.85\\linewidth]{gambar/hasil.png}\n
   \\end{center}\n\\begin{center}\n{\\small\\color{abuteks} Gambar 1: Keterangan}\n
   \\end{center}"
```

Untuk gambar yang dihasilkan kode R, tidak perlu perintah apa pun — keluaran grafik disisipkan otomatis

oleh knitr (dibahas Bab 8).

7.7 Karakter Khusus

Lima karakter berikut memiliki arti khusus dalam LaTeX dan harus diloloskan bila hendak dicetak apa adanya.

Karakter yang harus diloloskan.

Karakter	Ditulis	Arti khusus dalam LaTeX
_	`\_`	penurunan (subscript)
%	`\%`	awal komentar
$	`\$`	pembuka/penutup mode matematika
&	`\&`	pemisah kolom tabel
#	`\#`	penanda argumen makro

Pelolosan otomatis. Pada kunci `penjelasan` dan `interpretasi`, paket menjalankan fungsi `latex_prose_safe()` yang meloloskan kelima karakter tersebut secara otomatis dan mengubah `` `kode` `` menjadi `\texttt{kode}`. Jadi Anda dapat menulis `input$n` atau `50%` apa adanya — keduanya akan tercetak benar.

Pelolosan ini **tidak** berlaku pada blok LaTeX mentah yang Anda tulis sendiri. Di sana, pelolosan tetap menjadi tanggung jawab Anda.

7.8 Menyunting Preamble

Seluruh definisi kotak, warna, dan tata letak hidup di berkas `preamble.tex` yang dibangkitkan paket. Berkas ini **boleh disunting** untuk keperluan khusus.

7.8.1 Struktur preamble

Bagian-bagian `preamble.tex`.

Bagian	Isi
Paket	`fontspec`, `tcolorbox`, `tikz`, `titlesec`, dll.
Font	`\setmainfont`, `\setmonofont`
Warna	empat belas `\definecolor`
Penomoran	`\thesection`, penomor contoh dan kode
Gaya judul	`\titleformat`, `\titlespacing`
Kepala/kaki	`\fancyhead`, `\fancyfoot`
Kotak	empat belas `\newtcolorbox`
Sampul	makro `\coverpembatas`

7.8.2 Mengganti warna

Palet BPS terdiri atas empat warna pokok. Menggantinya cukup mengubah kode heksadesimalnya.

```
\definecolor{bpsnavy}{HTML}{123A6B}      % biru tua -- judul dan kepala halaman
\definecolor{bpsbiru}{HTML}{0B6FB4}      % biru -- aksen utama
\definecolor{bpsoranye}{HTML}{F58220}    % oranye -- penanda dan peringatan
\definecolor{bpshijau}{HTML}{00A651}     % hijau -- kunci jawaban
```

Perubahan pada satu tempat ini langsung berlaku pada seluruh kotak, sampul, dan penomoran.

7.8.3 Mengganti font

```
\setmainfont{Carlito}[Ligatures=TeX]
\setmonofont{DejaVu Sans Mono}[Scale=0.88]
```

Bila Carlito tidak tersedia, gantilah dengan font lain yang terpasang — misalnya `Liberation Sans` atau `TeX Gyre Heros`.

7.8.4 Menambah kotak baru

Kotak baru didefinisikan meniru pola kotak yang sudah ada.

```
\newtcolorbox{kotakcontoh}[1][Contoh Kasus]{
  breakable, enhanced,
  colback=bpshijau!5,          % warna latar (5 persen kepekatan)
  colframe=bpshijau,           % warna bingkai
  boxrule=0pt, leftrule=4pt,   % hanya garis kiri yang tebal
  arc=2pt,
  left=10pt, right=10pt, top=7pt, bottom=7pt,
  before upper={\textcolor{bpshijau!70!black}{\sffamily\bfseries #1.}~}}
```

Setelah didefinisikan, kotak dapat langsung dipakai dari YAML:

```
- "\\begin{kotakcontoh}[Kasus Survei]\nIsi kotak.\n\\end{kotakcontoh}"
```

Anatomi definisi kotak.

`[1][Judul]`	kotak menerima satu argumen opsional; `Judul` adalah bawaannya
`breakable`	kotak boleh terpotong antar-halaman
`colback`	warna latar; `warna!5` berarti kepekatan 5 persen
`colframe`	warna bingkai
`leftrule`	tebal garis kiri; pola khas modul ini
`before upper`	teks judul yang dicetak sebelum isi; `#1` adalah argumennya

Preamble akan tertimpa. Berkas `preamble.tex` pada folder modul **dibangkitkan ulang** setiap kali render. Penyuntingan langsung padanya akan hilang. Bila perubahan ingin permanen, suntinglah templatnya di dalam paket — `inst/templates/preamble.tex.tmpl` — lalu pasang ulang paketnya. Cara

ini dibahas lebih lanjut pada Bab 9.

7.9 Membaca Galat LaTeX

Pesan galat LaTeX terkenal sulit dibaca. Berikut yang paling sering muncul.

Galat LaTeX yang lazim.

Pesan	Penyebab dan tindakan
`Undefined control sequence`	perintah tidak dikenal — salah ketik, atau kotak belum didefinisikan
`Missing $ inserted`	karakter _ atau ^ di luar mode matematika; loloskan dengan _
`File `x.sty' not found`	paket LaTeX belum ada; jalankan `tinytex::tlmgr_install("x")`
`Extra alignment tab`	jumlah & pada baris tabel melebihi jumlah kolom yang dideklarasikan
`Runaway argument`	kurung kurawal tidak berpasangan
`pgfkeys: I do not know the key`	argumen opsional kotak keliru — periksa apakah kotak itu memang menerima judul

Tips dan Praktik Baik. Nomor baris pada pesan galat mengacu pada berkas `.tex` hasil bangkitan, bukan pada YAML Anda. Untuk menelusurinya, render dengan `keep_tex = TRUE` pada berkas `.Rmd`, lalu bukalah `.tex`-nya pada baris yang disebutkan.

Bab 8 Sintaks R Markdown dan Knitr

Bab ini membahas lapisan di bawah LaTeX: bagaimana kode dijalankan dan keluarannya disisipkan ke dalam dokumen.

8.1 Struktur Berkas yang Dibangkitkan

Berkas `.Rmd` hasil bangkitan memiliki tiga bagian.

Bagian berkas `.Rmd`	
Header YAML	menetapkan format keluaran, mesin LaTeX, dan preamble
Chunk `setup`	mengatur perilaku bawaan seluruh chunk
Isi	teks, blok LaTeX, dan chunk kode

```yaml
---
output:
  pdf_document:
    latex_engine: xelatex
    number_sections: true
    includes:
      in_header: preamble.tex
geometry: "left=2.6cm,right=2.4cm,top=3.0cm,bottom=2.4cm"
fontsize: 11pt
---
```

Catatan. Mesin `xelatex` wajib dipakai karena preamble memuat `fontspec` untuk font Carlito. Mesin `pdflatex` tidak mendukungnya.

8.2 Chunk Kode

Chunk adalah blok kode yang dijalankan saat render. Penulisannya diapit tiga tanda petik balik.

```
```r
x <- c(1, 2, 3)
mean(x)
```
```

8.2.1 Opsi chunk yang dipakai modul

Opsi chunk.

Opsi	Bawaan	Kegunaan
`echo`	`TRUE`	menampilkan kodenya
`eval`	`TRUE`	menjalankan kodenya
`include`	`TRUE`	`FALSE` menjalankan tanpa menampilkan apa pun
`warning`	`FALSE`	menampilkan peringatan
`message`	`FALSE`	menampilkan pesan
`comment`	`""`	awalan baris keluaran
`fig.width`	7	lebar gambar (inci)
`fig.height`	4.2	tinggi gambar (inci)
`out.width`	`"88%"`	lebar tampil relatif terhadap kolom
`fig.align`	`"center"`	perataan gambar

Opsi bawaan ditetapkan sekali pada chunk `setup`:

```
knitr::opts_chunk$set(
  echo = TRUE, warning = FALSE, message = FALSE, comment = "",
  fig.align = "center", fig.width = 7, fig.height = 4.2,
  out.width = "88%", fig.pos = "H"
)
options(width = 90, digits = 4, scipen = 999)
```

Tiga pengaturan `options` yang penting.

`width = 90`	lebar keluaran teks; mencegah baris terpotong di tepi h
`digits = 4`	banyaknya angka bermakna pada keluaran
`scipen = 999`	mencegah angka tercetak dalam notasi ilmiah

Tips dan Praktik Baik. Pengaturan `scipen = 999` sangat membantu pada modul statistik: tanpa itu, angka seperti 1500000 akan tercetak `1.5e+06` yang menyulitkan pembacaan mahasiswa.

8.3 Chunk Python

Modul dwibahasa menjalankan Python melalui paket `reticulate`.

```
```python
import pandas as pd
df = pd.read_csv("data/demo.csv")
print(df.head(3))
```
```

Python memerlukan `print`. Berbeda dari R yang mencetak nilai terakhir secara otomatis, Python hanya mencetak bila diminta. Bungkuslah dengan `print()` agar keluarannya muncul di modul.

8.3.1 Berbagi objek antar bahasa

Objek dapat dipertukarkan antara R dan Python dalam satu dokumen.

Mengakses objek lintas bahasa.		
	Dari R ke objek Python	`py$nama_objek`
	Dari Python ke objek R	`r.nama_objek`

```
library(reticulate)
mean(py$df$umur)      # membaca DataFrame Python dari R
```

8.4 Kait Keluaran

Keluaran kode di dalam modul tidak tampil polos, melainkan dibungkus kotak berwarna hijau muda. Ini dikerjakan oleh **kait keluaran** (*output hook*) pada chunk `setup`.

```
knitr::knit_hooks$set(output = function(x, options) {
  paste0("\\begin{routbox}\n\\begin{verbatim}\n", x,
         "\\end{verbatim}\n\\end{routbox}\n")
})
```

Kait ini menyadap setiap keluaran teks, lalu membungkusnya dengan lingkungan `routbox` yang didefinisikan di preamble. Karena bekerja pada tingkat knitr, seluruh chunk terpengaruh tanpa perlu perintah tambahan.

Catatan. Kait keluaran dipasang otomatis oleh paket. Anda tidak perlu menulisnya — pengetahuan ini berguna hanya bila hendak mengubah tampilan keluaran.

8.5 Menyisipkan Nilai Sebaris

Nilai hasil perhitungan dapat disisipkan langsung ke dalam kalimat.

```
Rata-rata pengeluaran adalah `r round(mean(x), 2)` ribu rupiah.
```

Cara ini membuat laporan **reproducible**: bila datanya berubah, angka dalam narasi ikut menyesuaikan tanpa penyuntingan manual.

Tips dan Praktik Baik. Anjurkan mahasiswa memakai penyisipan sebaris pada laporan penugasan mereka. Menyalin angka dengan tangan adalah sumber kekeliruan yang lazim, dan mudah dihindari.

8.6 Gambar dari Kode

Grafik yang dihasilkan kode disisipkan otomatis. Ukurannya dikendalikan dua kelompok opsi yang sering tertukar.

Dua kelompok ukuran gambar.

`fig.width`, `fig.height`	ukuran **kanvas** saat grafik digambar (inci); memengaruhi huruf relatif
`out.width`, `out.height`	ukuran **tampil** di dalam dokumen

Huruf grafik terlalu kecil. Bila huruf pada grafik tampak kekecilan, jangan memperbesar `out.width` — tindakan itu justru memperkecilnya secara relatif. Kecilkanlah `fig.width` sehingga kanvasnya lebih sempit dan hurufnya menjadi proporsional lebih besar.

8.7 Menjaga Hasil Tetap Reprodusibel

Tiga kebiasaan berikut menjamin modul menghasilkan angka yang sama setiap kali dirender.

Tiga kebiasaan.

Kunci pembangkit acak	`set.seed(2025)` sebelum kode yang memakai bilangan acak
Jalur relatif	`"data/berkas.csv"`, bukan jalur mutlak
Sebut versi paket	cantumkan versi paket pada bagian *Alat dan Bahan*

Kunci acak ditempatkan keliru. `set.seed()` hanya berpengaruh pada pembangkitan acak *sesudah*-nya. Bila diletakkan setelah kode yang membangkitkan data, hasilnya tetap berubah-ubah. Letakkan sebelum baris yang memerlukannya.

8.8 Menampilkan Kode tanpa Menjalankan

Tiga keadaan menuntut kode ditampilkan tanpa dijalankan; masing-masing memiliki cara sendiri.

Tiga cara.

Cara	Kapan dipakai
`eval: false` pada contoh	kode aplikasi Shiny, perintah pemasangan paket
Blok berpagar biasa	kerangka latihan yang harus dilengkapi mahasiswa
`echo = FALSE`	kode penyiapan yang tidak perlu dilihat mahasiswa

Untuk kerangka latihan, blok berpagar biasa lebih tepat daripada chunk, karena isinya memang tidak dimaksudkan sebagai kode yang sah.

```
- "```{r eval=FALSE}\nx <- pel$nilai\nmean(x)\n____(x)   # simpangan baku\n```"
```

8.9 Ringkasan Bab

Yang perlu diingat.

Mesin LaTeX	wajib `xelatex` karena memakai `fontspec`
Opsi bawaan	ditetapkan sekali pada chunk `setup`
Python	memerlukan `print()` agar keluarannya tampil
Lintas bahasa	`py$obj` dari R, `r.obj` dari Python
Kait keluaran	membungkus keluaran ke dalam kotak; dipasang otomatis
Ukuran gambar	`fig.*` mengatur kanvas, `out.*` mengatur tampilan
Reprodusibel	`set.seed`, jalur relatif, sebut versi paket

Bab 9 Menyesuaikan Tampilan

Bab ini membahas cara mengubah rupa modul: warna, font, kotak, dan sampul. Seluruh penyesuaian dilakukan pada lapisan **tema**, sehingga isi modul tidak perlu disentuh.

9.1 Tiga Tingkat Penyesuaian

Memilih tingkat penyesuaian.

Tingkat	Cara	Cakupan
Sekali pakai	sunting `preamble.tex` di folder modul	satu modul, hilang saat render u
Satu mata kuliah	salin templat, pakai `img_dir` sendiri	seluruh modul mata kuliah itu
Permanen	sunting `inst/templates/` lalu pasang ulang paket	seluruh mata kuliah

Penyuntingan sekali pakai akan hilang. Berkas `preamble.tex` pada folder modul **dibangkitkan ulang** setiap render. Menyuntingnya berguna untuk mencoba-coba, tetapi jangan menyimpan perubahan penting di sana.

9.2 Objek Tema

Tema dibuat dengan `theme_factory()` dan menyimpan seluruh keputusan tampilan.

```
th <- theme_factory()              # palet BPS bawaan
th$palette                         # daftar warna
th$preamble(number = 1, topic = "Pengenalan")   # menghasilkan preamble
th$box("tips", "Isi kotak")        # menghasilkan satu kotak
```

Metode objek tema.

`$palette`	daftar enam warna: `navy`, `biru`, `birumuda`, `oranye`, `hijau`, `abuteks`
`$preamble()`	membangkitkan seluruh isi `preamble.tex`
`$box()`	membangkitkan satu kotak berwarna
`$cover()`	membangkitkan sampul pembatas modul

9.3 Mengganti Palet Warna

Palet dapat diganti dengan menyerahkan daftar warna sendiri.

```
tema_kampus <- theme_factory(palette = list(
  navy     = "#0B1F3A",
  biru     = "#1D6FB8",
```

```
  birumuda = "#7FC0E8",
  oranye   = "#E8791B",
  hijau    = "#128A4B",
  abuteks  = "#3F4A54"
))

render_module(sp, theme = tema_kampus, output_dir = ".")
```

Perubahan berlaku serentak pada sampul, kotak, penomoran, dan kepala halaman — karena seluruhnya merujuk pada nama warna yang sama, bukan pada kode heksadesimal yang ditulis berulang.

Menyesuaikan program studi. Nama program studi tidak diatur lewat tema, melainkan lewat kunci `prodi` pada blok `course` sebuah spesifikasi. Nilainya tampil pada sampul pembatas dan kaki halaman, sehingga satu paket dapat melayani beberapa program studi tanpa perubahan templat.

Tips dan Praktik Baik. Sebelum mengganti palet, periksalah keterbacaannya pada cetakan hitam putih. Warna navy dan oranye pada palet BPS sengaja dipilih karena tetap terbedakan meski dicetak abu-abu.

9.4 Mengganti Font

Font ditetapkan pada templat preamble.

```
\setmainfont{Carlito}[Ligatures=TeX]
\setmonofont{DejaVu Sans Mono}[Scale=0.88]
```

Bila Carlito tidak tersedia di komputer sasaran, gantilah dengan font yang ada. Beberapa pilihan yang metriknya berdekatan:

Alternatif font.

`Carlito`	pilihan bawaan; metrik setara Calibri
`Liberation Sans`	tersedia luas di Linux; metrik setara Arial
`TeX Gyre Heros`	disertakan TeX Live, tidak perlu pemasangan terpisah
`DejaVu Sans`	selalu tersedia, tetapi lebih lebar sehingga tata letak bergeser

9.5 Menambah Kotak Baru Secara Permanen

Kotak yang sudah didefinisikan pada Bab 7 hanya berlaku sekali pakai. Agar permanen, definisinya ditaruh di templat paket.

1 — Buka templat. Sunting `inst/templates/preamble.tex.tmpl` pada sumber paket.

2 — Tambahkan definisi. Letakkan definisi kotak baru di dekat definisi kotak lain, agar mudah ditemukan kelak.

3 — Pasang ulang paket. Jalankan `R CMD build` lalu pasang tarballnya, atau muat ulang dengan `pkgload::load_all()` bila sedang mengembangkan.

4 — Tambahkan uji. Sertakan uji sederhana yang memastikan kotak baru muncul pada preamble, agar tidak hilang tanpa disadari saat penyuntingan berikutnya.

```
test_that("kotak baru tersedia di preamble", {
  pre <- theme_factory()$preamble(number = 1, topic = "Uji")
  expect_true(grepl("kotakcontoh", pre, fixed = TRUE))
})
```

9.6 Sampul Buku

Fungsi `sampul_buku()` membangkitkan sampul bergaya BPS/STIS — logo, latar gedung, dan pita diagonal — sebagai potongan LaTeX yang siap disisipkan.

```
sampul <- sampul_buku(
  judul1   = "PEMROGRAMAN",
  judul2   = "STATISTIKA",
  kode     = "K242310",
  semester = "3 (Ganjil)",
  penyusun = c("Achmad Syahrul Choir", "Rudi Salam"),
  jenis    = "BUKU MODUL PRAKTIKUM",
  ringkas  = "14 Modul Praktikum - Berbasis R dan Python",
  varian   = "modul"
)
cat(sampul, file = "sampul.tex")
```

Argumen `sampul_buku()`.

`judul1`, `judul2`	dua baris judul besar
`kode`	kode mata kuliah; tercetak oranye
`semester`	keterangan semester
`penyusun`	vektor nama; boleh berapa pun banyaknya
`jenis`	label di atas judul
`ringkas`	satu baris keterangan isi buku
`varian`	`"modul"` (pita biru) atau `"kunci"` (pita hijau)
`img_dir`	folder logo dan latar; bawaannya aset di dalam paket

Hasilnya disisipkan ke dalam berkas R Markdown melalui blok LaTeX mentah.

```
```{=latex}
```
```

> **Tips dan Praktik Baik.** Untuk memakai logo instansi lain, salinlah `logo_stis.png` dan `gedung_bg.png` ke folder Anda sendiri, gantilah berkasnya, lalu tunjuk foldernya melalui argumen `img_dir`. Ukuran gambar sebaiknya serupa agar tata letaknya tetap.

9.7 Menyesuaikan Sampul Pembatas Modul

Sampul pembatas tiap modul — halaman berwarna dengan nomor besar — dibangkitkan makro `\coverpembatas` di dalam preamble. Makro ini menerima empat argumen.

```
\coverpembatas{1}{PENGENALAN DAN INSTALASI}{Pertemuan Minggu ke-1}{Sub-CPMK 1: ...}
```

Untuk mengubah rupanya — misalnya menggeser posisi nomor atau mengganti bentuk hiasan — sunting definisi makro tersebut pada templat, lalu pasang ulang paket.

Bab 10 Menyusun Buku Gabungan

Modul yang sudah jadi dapat dirangkai menjadi satu buku. Bab ini membahas prosesnya, termasuk gagasan **penomoran halaman berlanjut**.

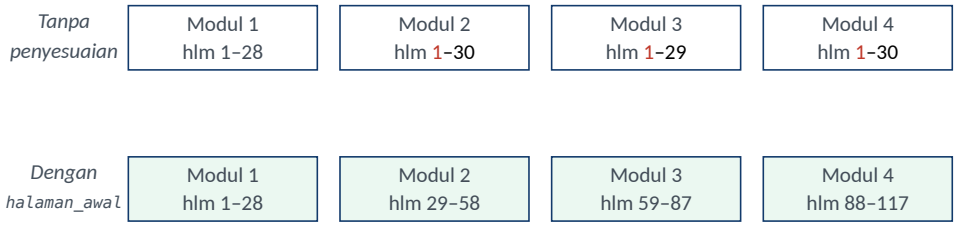
10.1 Susunan Sebuah Buku

Susunan baku.

Bagian awal	sampul, hak cipta, kata pengantar, petunjuk penggunaan, simbol dan kotak, penilaian, daftar isi — bernomor *romawi*
Isi	seluruh modul berurutan — bernomor *arab*, *berlanjut*
Bagian akhir	glosarium, kartu referensi, daftar pustaka gabungan, lampiran — melanjutkan nomor arab

10.2 Masalah Penomoran Halaman

Bila setiap modul dirender sendiri-sendiri, masing-masing memulai penomoran dari 1. Ketika dirangkai, buku memuat empat belas halaman bernomor "1" — daftar isi menjadi tak berguna dan pembaca tersesat.



Gambar 10.1: Penomoran halaman sebelum dan sesudah penyesuaian

10.3 Bagaimana Paket Menanganinya

Kunci `halaman_awal` pada blok `course` menetapkan nomor halaman pertama sebuah modul.

```
sp <- module_spec_from_yaml("modul-02.yml")
sp$course$halaman_awal <- 30      # modul ini dimulai pada halaman 30
render_module(sp, output_dir = ".")
```

Di dalam preamble, nilai ini menjadi `\setcounter{page}{30}`.

Letak `\setcounter` menentukan. Perintah `\setcounter{page}` harus ditempatkan **sesudah** `\end{titlepage}`. Lingkungan `titlepage` pada LaTeX mengembalikan penomor halaman ke 1 saat berakhir — sehingga penyetelan yang diletakkan di preamble akan terhapus tanpa pesan galat apa pun. Gejalanya: nomor halaman tetap dimulai dari 1 meski `halaman_awal` sudah diisi.

10.4 Alur Dua Tahap

Nomor awal sebuah modul bergantung pada jumlah halaman modul-modul sebelumnya — padahal jumlah itu baru diketahui setelah dirender. Karena itu prosesnya dua tahap.



Gambar 10.2: Alur dua tahap penomoran berlanjut

Fungsi `render_berlanjut()` mengerjakan keduanya sekaligus.

```
specs <- sort(Sys.glob("modul/*/modul-*.yml"))
dirs  <- dirname(specs)

hasil <- render_berlanjut(specs, dirs, jenis = "modul")
print(hasil)
```

```
                   spec hal mulai akhir
1  modul-01-pengenalan.yml  28     1    28
2       modul-02-dasar.yml  30    29    58
3 modul-03-pemrograman.yml  29    59    87
```

Argumen `render_berlanjut()`.

`specs`	vektor path berkas YAML, **terurut sesuai urutan buku**
`out_dirs`	folder keluaran, sepanjang `specs`
`jenis`	`"modul"` atau `"kunci"`
`theme`	objek tema; `NULL` memakai bawaan
`mulai`	nomor halaman pertama buku (bawaan 1)

Tips dan Praktik Baik. Karena tahap pertama merender seluruh modul, prosesnya memakan waktu dua kali lipat. Jalankan hanya ketika hendak menyusun buku — untuk menyunting satu modul, cukup render biasa.

10.4.1 Mengapa ditambah dua

Offset dihitung sebagai jumlah kumulatif halaman sebelumnya **ditambah dua**. Angka dua muncul karena halaman pertama tiap modul adalah sampul pembatas yang ikut terhitung tetapi nomornya tidak dicetak.

$$\text{offset}_i = \left(\sum_{j<i} \text{halaman}_j\right) + 2$$

Dengan begitu, sampul modul ke-i memperoleh nomor yang tepat meski tidak tercetak, dan isinya melanjutkan tanpa lompatan.

10.5 Menyusun Bagian Awal dan Akhir

Bagian awal dan akhir ditulis sebagai berkas R Markdown tersendiri, memakai preamble yang disesuaikan.

Penyesuaian preamble bagian awal.

Penomoran bagian	`\arabic{section}`, tanpa nomor modul
Nomor halaman	`\pagenumbering{roman}`
Kepala halaman	disesuaikan, mis. “Bagian Awal”
Judul bagian	memakai `{-}` agar tidak bernomor

Bagian akhir memakai pola serupa, tetapi melanjutkan penomoran arab:

```
\pagenumbering{arabic}
\setcounter{page}{457}
```

10.6 Menggabungkan Menjadi Buku

Setelah seluruh PDF siap, `gabung_pdf()` merangkainya.

```
berkas <- c(
  "buku/bagian_awal.pdf",
  sort(Sys.glob("modul/*/Modul_*.pdf")),
  "buku/bagian_akhir.pdf"
)
gabung_pdf(berkas, "Buku_Modul_Praktikum.pdf")
```

Urutan berkas menentukan. Fungsi `sort()` mengurutkan menurut abjad, sehingga `Modul_10` akan mendahului `Modul_2`. Pakailah penamaan berangka nol di depan (`Modul_02`) atau urutkan secara eksplisit menurut nomor modulnya.

```
# pengurutan yang benar menurut nomor modul
dirs <- list.dirs("modul", recursive = FALSE)
dirs <- dirs[order(as.integer(sub(".*Modul(\\d+)_.*", "\\1", basename(dirs))))]
```

10.7 Memeriksa Hasil

Tiga pemeriksaan berikut sebaiknya dijalankan sebelum buku diterbitkan.

```
B <- "Buku_Modul_Praktikum.pdf"

# 1) jumlah halaman sesuai harapan
pdf_halaman(B)

# 2) urutan modul benar -- periksa kepala halaman beberapa titik
```

```
system(sprintf("pdftotext -f 13 -l 13 %s - | head -1", shQuote(B)))

# 3) tidak ada galat kode yang lolos ke dalam PDF
system(sprintf("pdftotext %s - | grep -c 'Error in'", shQuote(B)))
```

Tips dan Praktik Baik. Pemeriksaan ketiga penting karena galat R yang terjadi saat render tidak selalu menghentikan prosesnya — pesan galat bisa ikut tercetak ke dalam PDF tanpa disadari.

Bab 11 Alur Kerja Sehari-hari

Bab penutup ini merangkum kebiasaan kerja yang membuat penyusunan modul berjalan lancar dalam jangka panjang.

11.1 Menyunting Satu Modul

Alur paling sering: mengubah isi satu modul lalu memeriksanya.

1. Sunting berkas `.yml` — bukan `.Rmd`.

2. Validasi: `module_spec_from_yaml("modul-01.yml")$validate()$ok`

3. Render: `bangun_modul("modul-01.yml")` — fungsi pintasan dari Bab 3.

4. Periksa PDF-nya, khususnya bagian yang baru disunting.

11.2 Membangun Seluruh Mata Kuliah

```
bangun_semua <- function(folder = "modul", jenis = c("modul", "kunci")) {
  dirs <- list.dirs(folder, recursive = FALSE)
  dirs <- dirs[order(as.integer(sub(".*Modul(\\d+)_.*", "\\1", basename(dirs))))]
  specs <- vapply(dirs, function(d) list.files(d, "\\.yml$", full.names = TRUE)[1],
                  character(1))

  # validasi seluruhnya lebih dahulu -- fail fast
  sah <- vapply(specs, function(s) module_spec_from_yaml(s)$validate()$ok, logical(1))
  if (!all(sah)) {
    stop("Spesifikasi belum sah: ", paste(basename(specs)[!sah], collapse = ", "))
  }

  for (j in jenis) render_berlanjut(specs, dirs, jenis = j)
}
```

Tips dan Praktik Baik. Memvalidasi seluruh spesifikasi *sebelum* render dimulai menghemat banyak waktu. Tanpa itu, kekeliruan pada modul keempat belas baru ketahuan setelah tiga belas modul selesai dirender.

11.3 Menaruh Sumber di Kendali Versi

Karena seluruh isi berupa teks, `git` bekerja dengan baik.

Yang dilacak dan yang diabaikan.

Dilacak	`*.yml`, `gen_data.R`, `README.md`, gambar yang dibuat
Diabaikan	`*.pdf`, `*.Rmd` hasil bangkitan, `preamble.tex`, `*.log`,
Pertimbangan	`data/*.csv` boleh dilacak agar hasil dapat direprodu atau diabaikan karena dapat dibangkitkan ulang

```
# .gitignore
*.pdf
Modul_*.Rmd
Kunci_*.Rmd
preamble*.tex
*.log
*.aux
*_files/
```

Jangan melacak berkas hasil bangkitan. Melacak `.Rmd` hasil bangkitan menimbulkan konflik terus-menerus, karena berkas itu berubah setiap render meski isinya sama. Lacaklah sumbernya saja.

11.4 Daftar Periksa Sebelum Menerbitkan

Sebelum modul dibagikan kepada mahasiswa:

- ☐ Seluruh spesifikasi lolos validasi
- ☐ Tidak ada pesan galat yang tercetak di dalam PDF
- ☐ Seluruh contoh memiliki Penjelasan Kode dan Interpretasi Output
- ☐ Latihan membetulkan kode disertai *Keluaran yang benar*
- ☐ Status data simulasi ditegaskan pada bagian Deskripsi
- ☐ Kunci jawaban dirender dari spesifikasi yang sama dengan modulnya
- ☐ Penomoran halaman berlanjut bila disusun sebagai buku
- ☐ Nama penyusun dan identitas mata kuliah sudah benar
- ☐ Berkas data tersedia dan dapat dibangkitkan ulang lewat `gen_data.R`

11.5 Memeriksa Mutu Secara Otomatis

Beberapa pemeriksaan dapat dijalankan sebagai skrip, sehingga tidak bergantung pada ketelitian mata.

```
audit_modul <- function(folder = "modul") {
  rmd <- list.files(folder, "^Modul_.*\\.Rmd$", recursive = TRUE, full.names = TRUE)
  for (f in rmd) {
    s <- readLines(f, warn = FALSE)
```

```
    teks <- paste(s, collapse = "\n")
    n_contoh    <- length(gregexpr("\\\\contoh\\{", teks)[[1]])
    n_penjelasan <- length(gregexpr("begin\\{kotakpenjelasan\\}", teks)[[1]])
    kosong      <- grepl("belum diisi", teks, fixed = TRUE)
    cat(sprintf("%-34s contoh=%2d penjelasan=%2d kosong=%s\n",
                basename(f), n_contoh, n_penjelasan, kosong))
  }
}
```

Tips dan Praktik Baik. Pemeriksaan otomatis paling berguna untuk hal yang mudah luput: bagian yang belum diisi, contoh tanpa penjelasan, atau istilah yang tidak baku. Untuk mutu isi, tetap diperlukan pembacaan manusia.

11.6 Bila Terjadi Masalah

Urutan penelusuran.

1. Apakah spesifikasi lolos validasi? Jalankan `$validate()`
2. Apakah galatnya dari R atau dari LaTeX? Pesan `Quitting from lines` menunjuk chunk R; pesan berawalan tanda seru menunjuk LaTeX
3. Bila dari R: jalankan kodenya di konsol secara terpisah
4. Bila dari LaTeX: render dengan `keep_tex = TRUE`, lalu buka `.tex` pada baris yang disebutkan
5. Apakah paket yang dimuat versi yang benar? Periksa dengan `find.package("modulstis")`

11.7 Penutup

Paket ini dibangun dari pengalaman menyusun dua mata kuliah: Analisis Peubah Ganda dengan dua puluh enam modul, dan Pemrograman Statistika dengan empat belas modul. Hampir setiap kotak *Kesalahan Umum* dalam buku ini berasal dari kekeliruan yang benar-benar terjadi selama penyusunan itu — kunci `n` yang terbaca sebagai boolean, penomor halaman yang direset lingkungan `titlepage`, dan versi paket lama yang menimpa sumber yang sedang disunting.

Gagasan pokoknya sederhana dan layak diulang di sini: **pindahkan sumber kebenaran dari naskah ke spesifikasi**. Sesudah itu, keseragaman, kesinkronan, dan kemudahan pemeliharaan mengikuti dengan sendirinya.

Mengembangkan lebih jauh. Paket ini dirancang terbuka untuk perluasan. Menambah jenis bagian baru cukup dengan mendaftarkan kelas `Section` baru; menambah backend render cukup dengan menurunkan kelas `Renderer`; menambah jenis data simulasi cukup dengan menambah tipe kolom pada `DataRecipe`. Ketiganya dibahas pada Lampiran C.

Lampiran A — Rujukan Cepat Spesifikasi YAML

Kunci Tingkat Atas

Kunci	Tipe	Wajib	Keterangan
`number`	integer	ya	nomor modul; menentukan penomoran `N.x`
`title`	teks	ya	judul lengkap modul
`short_code`	teks	tidak	potongan nama berkas keluaran
`topic`	teks	tidak	judul ringkas untuk kepala halaman
`pertemuan`	integer	tidak	pertemuan ke berapa pada RPS
`sub_cpmk`	teks	tidak	rumusan capaian pembelajaran
`packages`	daftar	tidak	paket yang dipakai modul
`contoh_title`	teks	tidak	judul bagian Contoh
`tambahan_title`	teks	tidak	judul bagian Tambahan
`course`	peta	tidak	identitas mata kuliah
`datasets`	daftar	tidak	resep data simulasi
`body`	peta	ya	isi kesebelas bagian
`exercises`	daftar	tidak	solusi latihan dan penugasan
`references`	daftar	tidak	daftar pustaka

Blok `course`

Kunci	Tipe	Keterangan
`matkul`	teks	nama mata kuliah
`matkul_besar`	teks	versi huruf kapital
`kode`	teks	kode mata kuliah
`semester`	teks	mis. `"3 (Ganjil)"`
`prodi`	teks	nama program studi (bawaan D-IV Komputasi Statistik)
`penyusun`	teks/daftar	satu atau beberapa nama
`halaman_awal`	integer	nomor halaman pertama modul

Blok `datasets`

Kunci	Tipe	Keterangan
`name`	teks	nama berkas tanpa akhiran; menjadi `data/nama.csv`
`role`	teks	`contoh` | `latihan` | `penugasan`
`recipe`	peta	resep pembangkitan (atau rujukan berkas nyata)
`kind`	teks	`tabular` | `mvn` | `grouped_mvn` | `categorical` | `latent_class` | `file`
`"n"`	integer	banyaknya baris — **harus berkutip**
`seed`	integer	kunci pembangkit acak
`columns`	peta	definisi tiap kolom

Memakai Data Nyata: `kind: file`

Sejak versi 0.3.4, dataset tidak harus disimulasikan. Dengan `kind: file`, berkas data yang sudah ada dipakai **apa adanya** — tanpa `seed`, tanpa pembangkitan.

Kunci	Tipe	Keterangan
`path`	teks	letak berkas; relatif terhadap **berkas YAML**, bukan direktori kerja
`columns`	daftar	(opsional) kolom yang dipakai sekaligus urutannya
`rename`	peta	(opsional) `nama_lama: nama_baru`
`"n"`	integer	(opsional) ambil n baris pertama
`na`	daftar	(opsional) penanda nilai hilang, mis. `["", "NA", "-"]`

Format yang didukung: `.csv`, `.csv2`, `.tsv`, `.txt`, dan `.rds`.

```yaml
datasets:
  - name: sosek_kabkota
    role: contoh
    recipe:
      kind: file
      path: data_asli/sosek_2024.csv
      columns: [kab, RLS, AHH, Sanitasi]
      rename: {AHH: harapan_hidup}
```

Berkas dibaca, kolom dipilih dan diganti nama bila diminta, lalu disalin menjadi `data/sosek_kabkota.csv` pada direktori modul *dan* kunci — persis seperti dataset simulasi, sehingga sisa modul tidak perlu berubah sama sekali.

Yang perlu diperhatikan. Data nyata membawa kewajiban yang tidak ada pada data simulasi: pastikan Anda berhak menyebarkannya, tidak ada satuan yang dapat diidentifikasi kembali, dan sumber serta tahun data dicantumkan pada bagian *Alat dan Bahan*. Bila data tidak boleh disebarkan bersama modul, letakkan berkasnya di luar repositori dan biarkan `path` menunjuk ke sana — `spec$validate()` akan melaporkan bila berkasnya tidak ditemukan.

Tipe Kolom

Tipe	Argumen	Menghasilkan
`numeric`	`min`, `max`, `digits`	pecahan seragam
`integer`	`min`, `max`	bilangan bulat
`category`	`levels`, `probs`	kategori acak
`balanced`	`levels`, `each`	kategori berimbang (faktorial)

Kesebelas Bagian `body`

N.1	`deskripsi`	apa isi modul dan mengapa penting
N.2	`tujuan`	capaian yang diharapkan
N.3	`alat`	perangkat lunak, paket, berkas data
N.4	`instruksi`	cara mengerjakan dan mengumpulkan
N.5	`teori`	landasan ringkas beserta rumus
N.6	`contoh`	contoh terpandu beserta kode
N.7	`latihan`	latihan berjenjang
N.8	`penugasan`	tugas terstruktur
N.9	`tambahan`	*opsional*; dilewati bila kosong
N.10	`ringkasan`	daftar periksa dan kartu referensi
N.11	`referensi`	diisi otomatis dari `references`

Kunci pada Sebuah Contoh

Kunci	Wajib	Keterangan
`contoh`	ya	judul; muncul sebagai *Contoh N.x*
`kode_r`	salah satu	kode R
`kode_py`	salah satu	kode Python
`judul_r`	tidak	label kotak kode R
`judul_py`	tidak	label kotak kode Python
`eval`	tidak	`true` menjalankan; `false` hanya menampilkan
`penjelasan`	tidak	isi kotak *Penjelasan Kode*
`interpretasi`	tidak	isi kotak *Interpretasi Output*

Kunci pada Sebuah Solusi

`id`	penanda, mis. `"L11.1"` (latihan) atau `"T11.1"` (penugasan)
`points`	bobot nilai
`solution`	ringkasan satu kalimat
`judul_r`	label kotak kode solusi
`solusi_r`	kode solusi R
`solusi_py`	kode solusi Python
`eval`	`false` bila solusi tidak dijalankan
`interpretasi`	alasan di balik solusinya

Lampiran B — Kartu Referensi LaTeX

Empat Belas Kotak

Lingkungan	Judul?	Judul bawaan / peran
`kotakperhatian`	ya	*Perhatian*
`kotaksalah`	ya	*Kesalahan Umum*
`kotaktips`	ya	*Tips dan Praktik Baik*
`kotakvariasi`	ya	*Variasi dan Alternatif*
`kotakcatatan`	ya	*Catatan*
`kotakinfo`	ya	tanpa judul bawaan
`kotakpenjelasan`	ya	*Penjelasan Kode* (otomatis)
`kotakinterpretasi`	ya	*Interpretasi Output* (otomatis)
`kotakanatomi`	ya	pasangan `ui-server`
`kotakrumus`	tidak	memuat rumus
`kotaktujuan`	tidak	daftar periksa penguasaan
`kotaktugas`	tidak	rumusan penugasan
`kotakkunci`	tidak	butir kunci jawaban
`kotakistilah`	tidak	daftar istilah

```
- "\\begin{kotaktips}[Judul Khusus]\nIsi kotak.\n\\end{kotaktips}"
```

Perintah Penomoran

Perintah	Hasil	Ditulis oleh
`\contoh{...}`	*Contoh N.x*	paket (otomatis)
`\kode{...}`	*Kode N.x*	paket (otomatis)
`\latihan{...}`	*Latihan N.x*	penyusun
`\tugas{...}`	*Tugas N.x*	penyusun

Empat Belas Warna

Nama	Heks	Dipakai untuk
`bpsnavy`	123A6B	judul, kepala halaman
`bpsbiru`	0B6FB4	aksen utama, kotak info
`bpsbirumuda`	6FB3E0	latar lembut
`bpsoranye`	F58220	penanda, kotak perhatian
`bpshijau`	00A651	kunci jawaban, kotak langkah
`abuteks`	45505C	teks sekunder
`codebg`	F4F7FB	latar kode R
`codebgpy`	FFF8E8	latar kode Python
`pykuning`	E8A200	aksen Python
`outbg`	F6FBF7	latar keluaran
`padanA`	C2410C	padanan masukan (RShiny)
`padanB`	1D4ED8	padanan keluaran
`padanC`	047857	padanan keluaran kedua
`merahsalah`	C0392B	penanda kekeliruan

Simbol Matematika

Perintah	Hasil	Perintah	Hasil
`\bar{x}`	$\bar{x}$	`\hat{p}`	$\hat{p}$
`\mu`	μ	`\sigma`	σ
`\alpha`	α	`\beta`	β
`\chi^{2}`	χ^2	`\eta^{2}`	η^2
`\sum`	$\sum$	`\sqrt{x}`	$\sqrt{x}$
`\dfrac{a}{b}`	$\dfrac{a}{b}$	`\leq`	$\leq$
`\neq`	$\neq$	`\times`	$\times$

Karakter yang Harus Diloloskan

Karakter	Ditulis	Arti khusus
_	`\_`	penurunan
%	`\%`	awal komentar
$	`\$`	mode matematika
&	`\&`	pemisah kolom
#	`\#`	argumen makro

Pada kunci `penjelasan` dan `interpretasi`, pelolosan dikerjakan otomatis oleh `latex_prose_safe()`.

Lampiran C — Rujukan API

Paket mengekspor **47 objek**: 19 kelas R6 dan 28 fungsi.

Fungsi yang Paling Sering Dipakai

`module_spec_from_yaml(path)`	memuat spesifikasi dari berkas YAML
`theme_factory(palette)`	membuat objek tema
`renderer_factory(backend, theme)`	membuat objek perender
`render_module(spec, ...)`	merender satu modul
`generate_module_data(spec, dir)`	membangkitkan data simulasi
`render_berlanjut(specs, dirs, ...)`	merender banyak modul berpenomoran berlanjut
`sampul_buku(...)`	membangkitkan sampul buku
`gabung_pdf(berkas, keluaran)`	menggabungkan PDF menjadi buku
`pdf_halaman(path)`	membaca jumlah halaman PDF

Fungsi Pendukung

`module_spec(...)`	membuat spesifikasi langsung dari R (tanpa YAML)
`generate_dataset(recipe)`	membangkitkan satu data.frame dari resep
`default_sections(...)`	daftar kesebelas bagian baku
`section_types()`	jenis bagian yang tersedia
`make_section(...)`	membuat objek bagian
`bps_palette()`	palet warna BPS
`build_logger(...)`	membuat pencatat proses
`latex_prose_safe(x)`	meloloskan karakter khusus LaTeX
`esc_latex(x)`, `esc_underscore(x)`	pelolosan versi ringkas
`load_glossary()`, `filter_glossary()`	memuat dan menyaring daftar istilah
`glossary_latex()`	membangkitkan daftar istilah sebagai LaTeX
`new_module_rmd()`, `new_answerkey_rmd()`	membuat kerangka Rmd kosong
`addin_new_module()` dll.	pintasan RStudio Addins

Kelas R6

Kelas	Peran
`ModuleSpec`	menyimpan seluruh isi spesifikasi; menyediakan `$validate()`
`ValidationReport`	hasil validasi: `$ok` dan `$messages`
`Section`	induk seluruh jenis bagian
`ExampleSection`	bagian Contoh dan Tambahan
`ExerciseSection`	bagian Latihan
`AssignmentSection`	bagian Penugasan
`GlossarySection`	bagian daftar istilah
`ContentBuilder`	induk pembangun dokumen
`ModuleBuilder`	membangun dokumen modul
`AnswerKeyBuilder`	membangun dokumen kunci jawaban
`Document`	dokumen hasil bangunan
`Renderer`	induk perender
`BookdownRenderer`	perender berbasis bookdown/rmarkdown
`QuartoRenderer`	perender berbasis Quarto
`RenderContext`	konteks penomoran saat render
`Artifact`	berkas hasil render
`LatexTheme`	tema: warna, preamble, kotak, sampul
`ModuleProject`	proyek berisi banyak modul
`Logger`	pencatat proses

Memperluas Paket

Menambah jenis bagian	turunkan `Section`, daftarkan pada `section_types()`, tambahkan ke `default_sections()`
Menambah backend render	turunkan `Renderer`, daftarkan pada `renderer_factory()`
Menambah tipe kolom data	tambahkan cabang pada `generate_dataset()` di `R/data-recipe.R`
Menambah kotak	tambahkan `\newtcolorbox` pada `inst/templates/preamble.tex.tmpl`

Setiap perluasan sebaiknya disertai uji pada `tests/testthat/`.

Lampiran D — Pemecahan Masalah

Galat Pemasangan

Gejala	Tindakan
`xelatex not found`	pasang TinyTeX; jalankan ulang RStudio sesudahnya
`File `tcolorbox.sty' not found`	`tinytex::tlmgr_install("tcolorbox")`
`font "Carlito" cannot be found`	pasang font Carlito, atau ganti `\setmainfont` pada preamble
`pdfunite: command not found`	pasang poppler-utils; hanya menghalangi buku gabungan
`there is no package called 'R6'`	pasang dependensi paket
Kata terpenggal keliru	pola pemenggalan Indonesia belum terpasang; lihat Bab 2

Galat Render yang Sukar Ditelusuri

Tiga galat berikut memiliki ciri yang sama: pesannya tidak menunjuk penyebab sebenarnya, sehingga mudah menyesatkan.

Gejala	Sebab dan tindakan
`Misplaced alignment tab character &`	judul memuat `&` yang belum diloloskan; sejak versi 0.3.1 paket meloloskannya sendiri
`Missing $ inserted`	ringkasan solusi memuat `$` atau `^` — misalnya `obj$kolom` atau `rho^2`; sejak 0.3.1 diloloskan otomatis
`could not find function "."`	sintaks `data.table` dipakai di dalam modul; sejak 0.3.1 paket menandai dirinya *datatable aware*
`Missing number, treated as zero`	penanda `@@HALAWAL@@` belum tergantikan; render melalui paket atau skrip `render_ulang.R`, bukan memanggil `.Rmd` langsung
`You may need to add $ $ around a certain inline R expression`	pesan ini *tidak* menunjuk ekspresi R inline: judul label memuat `^` atau `~` — misalnya `\kode{Arah pembeda Sp^{-1}}`; sejak 0.3.2 keempat label meloloskannya sendiri

Menulis rumus pada judul label. Sejak versi 0.3.2, bagian judul yang diapit `$...$` dibiarkan apa adanya sehingga tercetak sebagai matematika, sedangkan di luarnya `&`, `_`, `%`, `#`, `^`, dan `~` diloloskan otomatis. Karena itu tulislah `\kode{Arah pembeda $\mathbf{S}_p^{-1}$}` — bukan `Sp^{-1}` telanjang, yang kini memang tidak lagi menghentikan kompilasi tetapi tercetak sebagai teks biasa.

Peubah tertimpa saat merender berkelompok. Bila Anda menulis skrip yang merender banyak modul dalam satu perulangan, berikan `envir = new.env()` pada `rmarkdown::render()`. Tanpa itu, kode di dalam modul berjalan pada lingkungan pemanggil dan dapat menimpa peubah skrip Anda — termasuk peubah perulangan. Gejalanya membingungkan: laporan menyebut modul yang keliru meskipun berkas yang dihasilkan sebenarnya benar.

Galat Spesifikasi

Gejala	Tindakan
Data selalu 100 baris	kunci `n` belum diberi tanda kutip; tulis `"n": 60`
Galat pembacaan YAML	indentasi memakai tab; ganti dengan spasi
Teks terpotong di titik dua	bungkus nilai dengan tanda kutip ganda
Kode kehilangan baris	memakai `>` untuk kode; ganti dengan `|`
`$validate()$ok` bernilai FALSE	baca `$messages` — seluruh kekeliruan dilaporkan sekaligus

Galat Render

Gejala	Tindakan
`Quitting from lines ...`	galat berasal dari chunk R; jalankan kodenya terpisah
`Undefined control sequence`	perintah LaTeX salah ketik atau kotak belum didefinisikan
`Missing $ inserted`	karakter _ di luar mode matematika
`Extra alignment tab`	jumlah `&` melebihi kolom yang dideklarasikan
`pgfkeys: I do not know the key`	kotak diberi argumen judul padahal tidak menerimanya
`object 'x' not found`	contoh memakai peubah yang didefinisikan pada bagian yang dirender belakangan

Galat Hasil

Gejala	Tindakan
Nomor halaman selalu mulai 1	`\setcounter{page}` diletakkan sebelum `\end{titlepage}`
Judul kotak tidak muncul	kotak itu tidak menerima argumen judul (lihat Lampiran B)
`**tebal**` tercetak apa adanya	Markdown tidak berlaku di dalam kotak; pakai `\textbf{}`
Huruf grafik terlalu kecil	kecilkan `fig.width`, jangan besarkan `out.width`
Angka tercetak `1.5e+06`	tambahkan `options(scipen = 999)`
Bagian Tambahan tidak muncul	`body$tambahan` kosong; bagian opsional memang dilewati
Perubahan paket tak berpengaruh	versi lama terpasang menimpa sumber; pakai `pkgload::load_all()`
Urutan modul kacau di buku	`sort()` menempatkan `Modul_10` sebelum `Modul_2`; urutkan menurut nomor

Urutan Penelusuran

1. Jalankan `$validate()`. Sebagian besar masalah tertangkap di sini.

2. Tentukan asal galat: `Quitting from lines` menunjuk R; pesan berawalan tanda seru menunjuk LaTeX.

3. Bila dari R — jalankan kodenya di konsol secara terpisah.

4. Bila dari LaTeX — render dengan `keep_tex = TRUE`, buka `.tex` pada baris yang disebutkan.

5. Periksa versi paket yang termuat: `find.package("modulstis")`.

Lampiran E — Studi Kasus: Mata Kuliah Baru

Lampiran ini menyusun satu mata kuliah baru dari nol: **Metode Survei Sampel**, tiga modul, lengkap sampai buku gabungan.

Menyiapkan Struktur

```r
dir.create("survei/modul", recursive = TRUE)
for (i in 1:3) dir.create(sprintf("survei/modul/Modul%02d", i))
```

```
survei/
  modul/
    Modul01/  Modul02/  Modul03/
  buku/
  bangun.R
```

Menulis Spesifikasi Pertama

```yaml
# survei/modul/Modul01/modul-01-kerangka.yml
number: 1
title: "Kerangka Sampel dan Populasi Sasaran"
short_code: "kerangka"
topic: "Kerangka Sampel"
pertemuan: 1
sub_cpmk: >
  Sub-CPMK 1: Mahasiswa mampu menjelaskan kerangka sampel dan
  mengevaluasi mutunya.
packages: [dplyr]

course:
  matkul: "Metode Survei Sampel"
  matkul_besar: "METODE SURVEI SAMPEL"
  kode: "K242320"
  semester: "4 (Genap)"
  prodi: "D-IV Statistika"
  penyusun:
    - "Nama Penyusun"

datasets:
  - name: data_kerangka
    role: contoh
    recipe:
      kind: tabular
      "n": 120
```

```yaml
      seed: 2025
      columns:
        kode_bs:  {type: integer, min: 1001, max: 1120}
        wilayah:  {type: category, levels: [Perkotaan, Perdesaan], probs: [0.45, 0.55]}
        rumah_tangga: {type: integer, min: 60, max: 180}

body:
  deskripsi:
    - >
      Modul ini membahas kerangka sampel: apa itu, bagaimana menilai
      mutunya, dan akibat kekeliruan kerangka terhadap penduga.
    - "\\begin{kotakperhatian}[Tentang data]\nBerkas data pada modul ini
       adalah \\textbf{data simulasi}, bukan data resmi.\n\\end{kotakperhatian}"
  tujuan:
    - "Setelah menyelesaikan praktikum ini, mahasiswa mampu:"
    - "1. **Menjelaskan** perbedaan populasi sasaran dan kerangka sampel."
    - "2. **Mengenali** cakupan lebih dan cakupan kurang."
  alat:
    - "- **R** (RStudio); paket `dplyr`."
    - "- Berkas data: `data/data_kerangka.csv`."
  instruksi:
    - "**Sebelum praktikum:** siapkan direktori kerja dan berkas data."
    - "**Selama praktikum:** kerjakan Contoh berurutan."
  teori:
    - "### Kaitan dengan Materi Sebelumnya"
    - >
      Modul ini memakai keterampilan manipulasi data untuk memeriksa
      mutu kerangka.
    - "### Populasi Sasaran dan Kerangka"
    - >
      Populasi sasaran adalah himpunan yang ingin disimpulkan, sedangkan
      kerangka adalah daftar yang benar-benar tersedia.
  contoh:
    - contoh: "Memeriksa mutu kerangka"
      kode_r: "kr <- read.csv(\"data/data_kerangka.csv\")\nstr(kr)\nsum(duplicated(kr$kode_bs))"
      judul_r: "str dan duplicated"
      eval: true
      penjelasan: >
        Data dibaca lalu strukturnya diperiksa. Fungsi duplicated menandai
        kode yang muncul lebih dari sekali, dan sum menghitungnya.
      interpretasi: >
        Kode ganda menandakan cakupan lebih: satu unit berpeluang terpilih
        dua kali, sehingga penduga menjadi berbias.
```

```
  latihan:
    - "#### Tingkat Dasar"
    - "\\latihan{Memeriksa keunikan}"
    - "```{r eval=FALSE}\nsum(____(kr$kode_bs))   # cacah kode ganda\n```"
  penugasan:
    - "\\begin{kotaktugas}\nEvaluasi mutu kerangka sampel.\n\\end{kotaktugas}"
    - "\\tugas{Pemeriksaan}"
    - "1. Periksa keunikan kode dan laporkan temuannya."
  ringkasan:
    - "\\begin{kotaktujuan}\n\\begin{itemize}
       \n\\item[$\\square$] Saya dapat membedakan populasi sasaran dan kerangka.
       \n\\end{itemize}\n\\end{kotaktujuan}"
    - "Modul berikutnya membahas penarikan sampel acak sederhana."

exercises:
  - id: "L1.1"
    points: 25
    solution: "Memakai duplicated untuk mencacah kode ganda."
    judul_r: "Solusi Tingkat Dasar (R)"
    solusi_r: |
      kr <- read.csv("data/data_kerangka.csv")
      sum(duplicated(kr$kode_bs))
    interpretasi: >
      Fungsi duplicated mengembalikan nilai logika; sum menghitung yang bernilai benar.

references:
  - "Cochran, W. G. (1977). *Sampling Techniques* (3rd ed.). Wiley."
```

Membangun Modul Pertama

```
library(modulstis)

sp <- module_spec_from_yaml("survei/modul/Modul01/modul-01-kerangka.yml")
sp$validate()$ok

generate_module_data(sp, output_dir = "survei/modul/Modul01")

th <- theme_factory()
r  <- renderer_factory("bookdown", theme = th)
r$render(ModuleBuilder$new(theme = th)$build(sp),    "survei/modul/Modul01")
r$render(AnswerKeyBuilder$new(theme = th)$build(sp), "survei/modul/Modul01")
```

Menambah Modul Berikutnya

Salin spesifikasi pertama sebagai kerangka, lalu ubah `number`, `title`, dan isinya. Blok `course` disalin apa adanya karena identitas mata kuliahnya sama.

```
file.copy("survei/modul/Modul01/modul-01-kerangka.yml",
          "survei/modul/Modul02/modul-02-srs.yml")
```

Membangun Seluruh Mata Kuliah

```
dirs  <- list.dirs("survei/modul", recursive = FALSE)
dirs  <- dirs[order(as.integer(sub(".*Modul(\\d+)", "\\1", basename(dirs))))]
specs <- vapply(dirs, function(d) list.files(d, "\\.yml$", full.names = TRUE)[1],
                character(1))

for (i in seq_along(specs)) {
  generate_module_data(module_spec_from_yaml(specs[i]), output_dir = dirs[i])
}

hasil_modul <- render_berlanjut(specs, dirs, jenis = "modul")
hasil_kunci <- render_berlanjut(specs, dirs, jenis = "kunci")
print(hasil_modul)
```

Menyusun Buku

```
sampul <- sampul_buku(
  judul1   = "METODE SURVEI",
  judul2   = "SAMPEL",
  kode     = "K242320",
  semester = "4 (Genap)",
  penyusun = "Nama Penyusun",
  ringkas  = "3 Modul Praktikum - Berbasis R",
  varian   = "modul"
)
cat(sampul, file = "survei/buku/sampul.tex")

# setelah bagian awal dan akhir dirender:
gabung_pdf(
  c("survei/buku/bagian_awal.pdf",
    file.path(dirs, list.files(dirs, "^Modul_.*\\.pdf$")),
    "survei/buku/bagian_akhir.pdf"),
  "survei/Buku_Metode_Survei_Sampel.pdf"
)
```

Daftar Periksa Penutup

Mata kuliah baru siap dipakai bila:

- ☐ Seluruh spesifikasi lolos `$validate()`
- ☐ Setiap contoh memiliki Penjelasan Kode dan Interpretasi Output
- ☐ Setiap modul memuat penegasan status data simulasi
- ☐ Kunci jawaban dirender dari spesifikasi yang sama
- ☐ Penomoran halaman berlanjut pada buku gabungan
- ☐ Tidak ada pesan galat yang tercetak di dalam PDF
- ☐ Seluruh sumber tersimpan di kendali versi

Glosarium

Istilah teknis yang dipakai dalam buku ini beserta padanan Inggrisnya.

Istilah	Inggris	Keterangan
bagian	*section*	satu dari sebelas bagian baku modul
bagian opsional	*optional section*	bagian yang dilewati bila tidak diisi (`tambahan`)
chunk	*chunk*	blok kode yang dijalankan saat render
kait keluaran	*output hook*	fungsi yang menyadap dan membungkus keluaran kode
kelas R6	*R6 class*	kelas berorientasi objek dengan rujukan; dasar rancangan paket
kotak	*box*	lingkungan LaTeX berwarna, mis. `kotaktips`
offset halaman	*page offset*	nomor halaman pertama sebuah modul dalam buku
pelolosan	*escaping*	menuliskan karakter khusus agar tercetak apa adanya
pembangkit acak	*random number generator*	dikunci dengan `set.seed` agar hasilnya sama
penomoran berlanjut	*continuous pagination*	nomor halaman berjalan terus lintas modul
perender	*renderer*	objek yang menjalankan mesin render (bookdown/Quarto)
preamble	*preamble*	bagian LaTeX sebelum isi dokumen; memuat definisi
resep data	*data recipe*	deskripsi deklaratif cara membangkitkan data simulasi
reprodusibel	*reproducible*	dapat dijalankan ulang dengan hasil yang sama
sampul pembatas	*divider page*	halaman berwarna pembuka tiap modul
spesifikasi	*specification*	berkas YAML berisi seluruh isi modul
sumber kebenaran	*single source of truth*	satu tempat yang menjadi acuan seluruh keluaran
tema	*theme*	objek yang menyimpan keputusan tampilan
templat	*template*	berkas pola yang diisi paket saat render
tipe kolom	*column type*	jenis data yang dibangkitkan resep

Istilah	Inggris	Keterangan
validasi	*validation*	pemeriksaan kesahihan spesifikasi sebelum render

Indeks Perintah

Fungsi Paket

Perintah	Bab
`AnswerKeyBuilder$new()$build()`	3, 11
`default_sections()`	5, Lamp. C
`gabung_pdf()`	10, Lamp. C
`generate_dataset()`	Lamp. C
`generate_module_data()`	3, Lamp. E
`latex_prose_safe()`	7, Lamp. C
`module_spec_from_yaml()`	3, 4, 11
`ModuleBuilder$new()$build()`	3, 11
`pdf_halaman()`	10, Lamp. C
`render_berlanjut()`	10, Lamp. E
`render_module()`	9, 10
`renderer_factory()`	3, Lamp. C
`sampul_buku()`	9, Lamp. E
`theme_factory()`	3, 9
`$validate()`	3, 4, 11

Kunci YAML

Kunci	Bab
`body`	4, 5
`contoh`, `kode_r`, `kode_py`	6
`course`	4, 9
`datasets`, `recipe`, `columns`	4
`eval`	6, 8
`exercises`	6
`halaman_awal`	4, 10
`interpretasi`	6, 7
`number`, `title`, `short_code`	4
`penjelasan`	6, 7
`penyusun`	4, 9
`references`	4
`tambahan`, `tambahan_title`	5

Lingkungan LaTeX

Lingkungan	Bab
`kotakanatomi`	7, Lamp. B
`kotakcatatan`	7, Lamp. B
`kotakinfo`	7, Lamp. B
`kotakinterpretasi`	6, 7
`kotakkunci`	7
`kotakpenjelasan`	6, 7
`kotakperhatian`	7, Lamp. B
`kotakrumus`	7, Lamp. B
`kotaksalah`	7, Lamp. B
`kotaktips`	7, Lamp. B
`kotaktugas`	5, 7
`kotaktujuan`	5, 7
`kotakvariasi`	7, Lamp. B
`\contoh{}`, `\kode{}`	7
`\latihan{}`, `\tugas{}`	5, 6, 7

Opsi Chunk

Opsi	Bab
`comment`	8
`echo`	8
`eval`	6, 8
`fig.width`, `fig.height`	8
`include`	8
`out.width`	8
`warning`, `message`	8

Daftar Pustaka

Rujukan yang dipakai dalam penyusunan paket dan buku ini.

Allaire, J. J., dkk. (2024). *rmarkdown: Dynamic Documents for R*. R package documentation.

Chang, W. (2021). *R6: Encapsulated Classes with Reference Semantics*. R package documentation.

Wickham, H. (2019). *Advanced R* (2nd ed.). Chapman and Hall/CRC.

Wickham, H., & Bryan, J. (2023). *R Packages* (2nd ed.). O'Reilly Media.

Xie, Y. (2015). *Dynamic Documents with R and knitr* (2nd ed.). Chapman and Hall/CRC.

Xie, Y. (2016). *bookdown: Authoring Books and Technical Documents with R Markdown*. Chapman and Hall/CRC.

Xie, Y., Allaire, J. J., & Grolemund, G. (2018). *R Markdown: The Definitive Guide*. Chapman and Hall/CRC.

Sturm, T. F. (2024). *The tcolorbox package: Coloured boxes for LaTeX examples and theorems*. CTAN documentation.

Tantau, T. (2023). *The TikZ and PGF Packages: Manual for version 3.1.10*. CTAN documentation.

Robertson, W., & Hosny, K. (2023). *The fontspec package: Font selection for XeLaTeX and LuaLaTeX*. CTAN documentation.

Tentang paket yang dijelaskan. Paket `modulstis` versi 0.3.7 dikembangkan untuk keperluan penyusunan modul praktikum di Politeknik Statistika STIS. Paket berisi 47 objek terekspor — 19 kelas R6 dan 28 fungsi — beserta 28 spesifikasi contoh dari dua program studi yang telah disusun dengannya. Seluruh sumber tersedia beserta berkas pendamping buku ini.

Buku ini disusun menggunakan `modulstis` 0.3.7, R Markdown, dan XeLaTeX.
Politeknik Statistika STIS · Jakarta · 5 September 2026